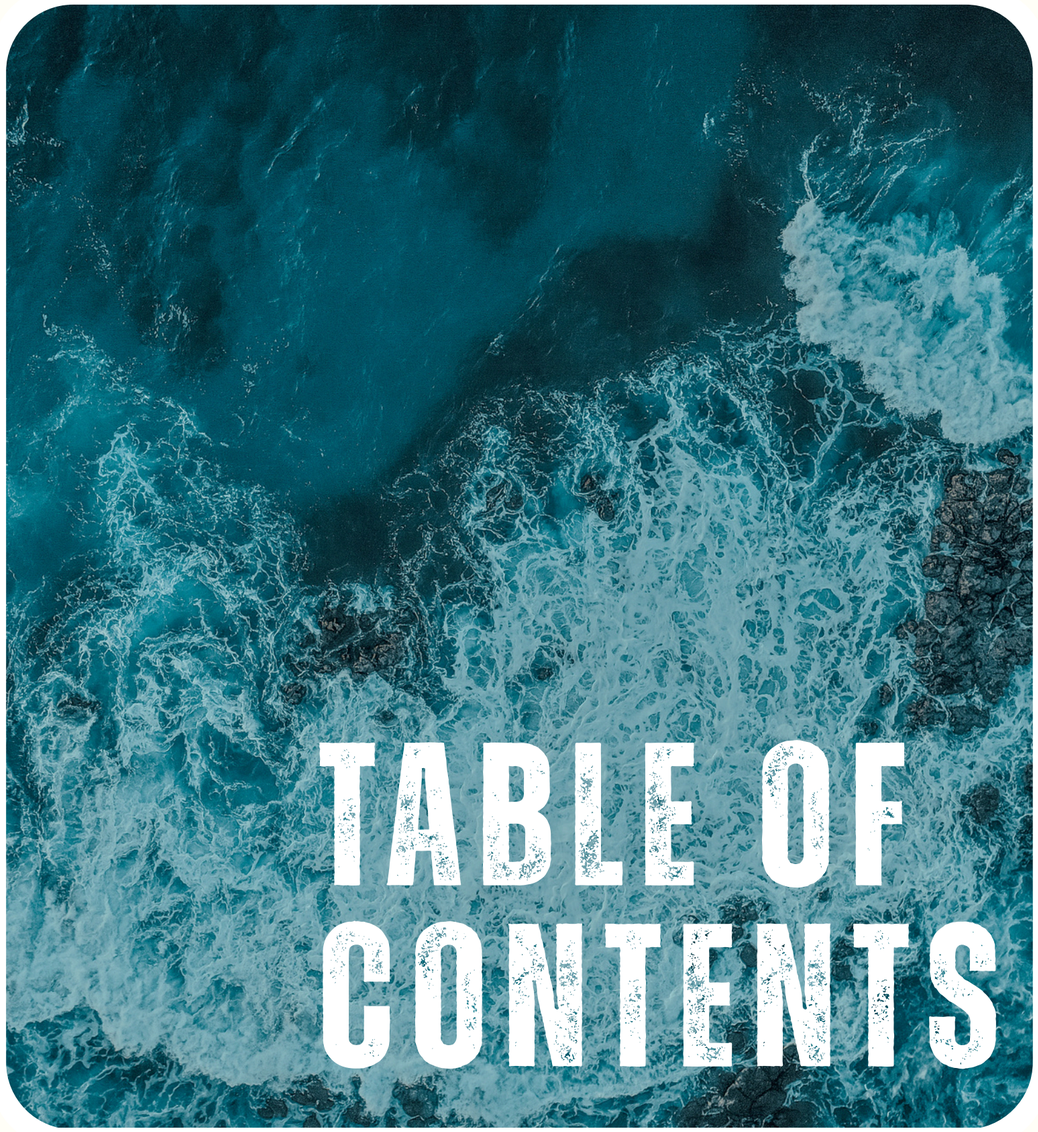
AQUASENTINEL

TUNISIA

SURVEILLER L'EAU, DÉTECTER LES FUITES, AGIR VITE

Projet AWD



1. Idée et problématique

le stress hydrique et les fuites découvertes trop tard

2. Acteurs et fonctionnalités

les acteurs, les cas d'utilisation et les principales fonctionnalités

3. Architecture globale

API Gateway, sécurité Keycloak, données et communications

4. Microservices et technologies

7 services, leurs bases et la technologie avancée GraphQL

5. Organisation du projet

la répartition, Git, le planning et les risques

6. Présentation des microservices

service commun, puis chaque microservice

PROBLÉMATIQUE:

- La Tunisie est en stress hydrique : sécheresses répétées, restrictions et coupures d'eau.
- Dans un immeuble, un hôtel ou un établissement, une fuite est souvent découverte tard, au moment de la facture.
- Les gestionnaires n'ont pas d'outil simple pour :
  - suivre la consommation de chaque bâtiment ;
  - repérer les consommations anormales ;
  - faire réparer rapidement avec les bonnes pièces ;
  - mesurer l'économie réalisée après réparation.

IDÉE GÉNÉRALE:

AQUASENTINEL TUNISIA EST UNE PLATEFORME WEB DE SUIVI DE LA CONSOMMATION D'EAU POUR LES HABITATIONS, IMMEUBLES, HÔTELS ET ÉTABLISSEMENTS.

LE CYCLE COMPLET :
RELEVÉ → FUITE DÉTECTÉE → INTERVENTION (TECHNICIEN + PIÈCE) → FACTURE ET ÉCONOMIES

LES RELEVÉS VIENNENT D'UN SIMULATEUR POUR LA DÉMONSTRATION, PUISQU'IL N'Y A PAS DE VRAI COMPTEUR CONNECTÉ.

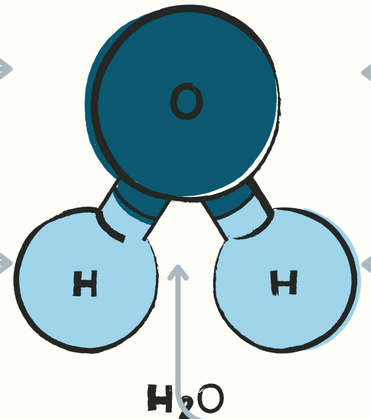
ACTEURS

CONSOMMATEUR

- Consulte la consommation et les factures de son bâtiment

GESTIONNAIRE

- Gère ses bâtiments, suit les relevés, vérifie les fuites, suit les interventions et les factures



TECHNICIEN

- Reçoit ses interventions, utilise les pièces réservées, rédige le compte rendu

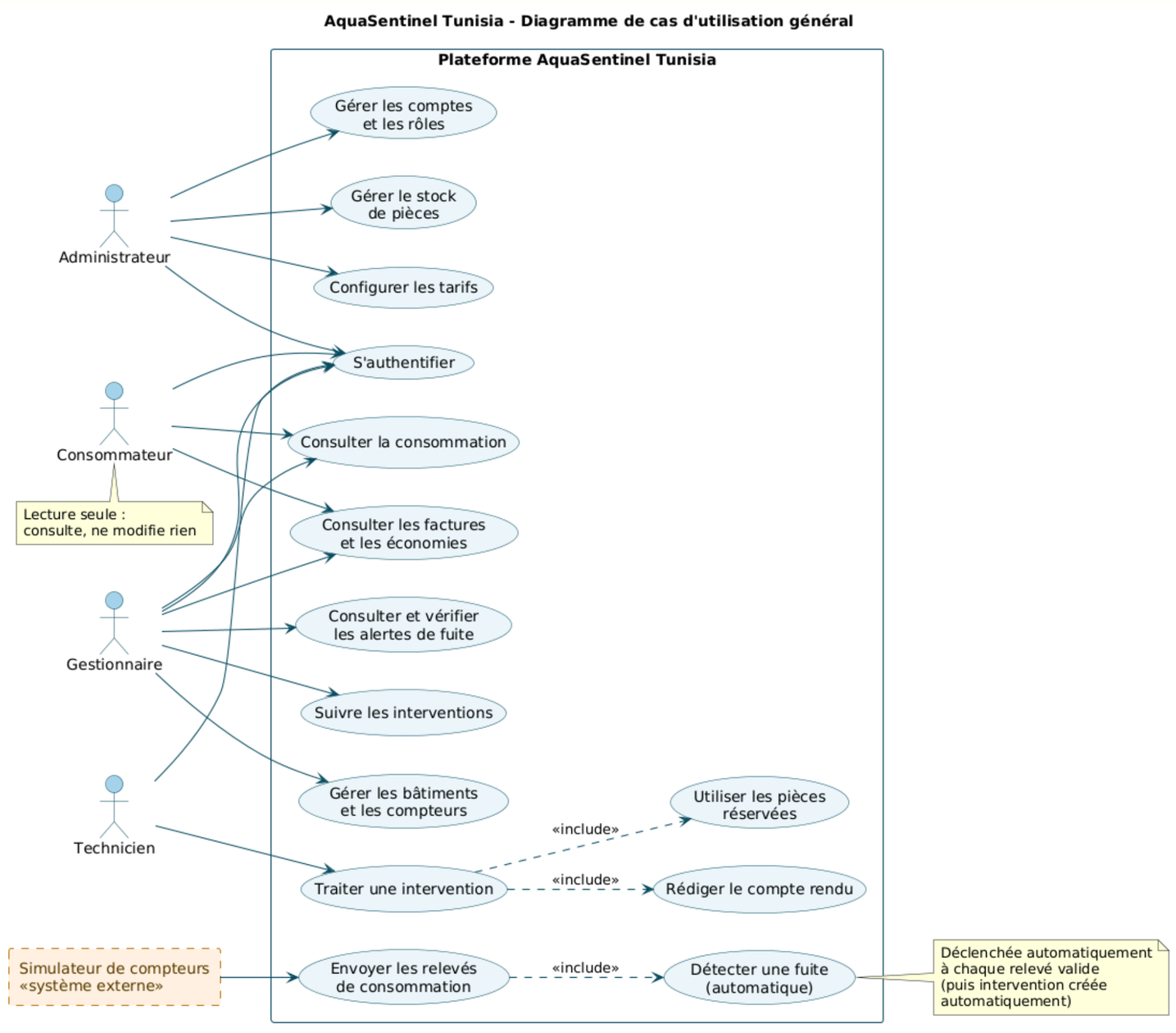
ADMINISTRATEUR

- Gère les comptes et les rôles, le stock de pièces et les tarifs

SIMULATEUR DE COMPTEURS

- Envoie automatiquement des relevés de consommation à la plateforme

DIAGRAMME DE CAS D'UTILISATION GÉNÉRAL:



PRINCIPALES FONCTIONNALITÉS:

1. BÂTIMENTS ET COMPTES

GÉRER LES BÂTIMENTS, LES COMPTEURS ET LES ACCÈS SELON LE RÔLE.

2. SUIVI DE LA CONSOMMATION

ENREGISTRER LES RELEVÉS, CONSULTER L'HISTORIQUE ET LES STATISTIQUES.

3. DÉTECTION DES FUITES

REPÉRER LES ANOMALIES, ÉVALUER LE RISQUE ET ESTIMER LA PERTE.

4. INTERVENTIONS

CRÉER AUTOMATIQUEMENT UNE INTERVENTION ET AFFECTER UN TECHNICIEN DISPONIBLE.

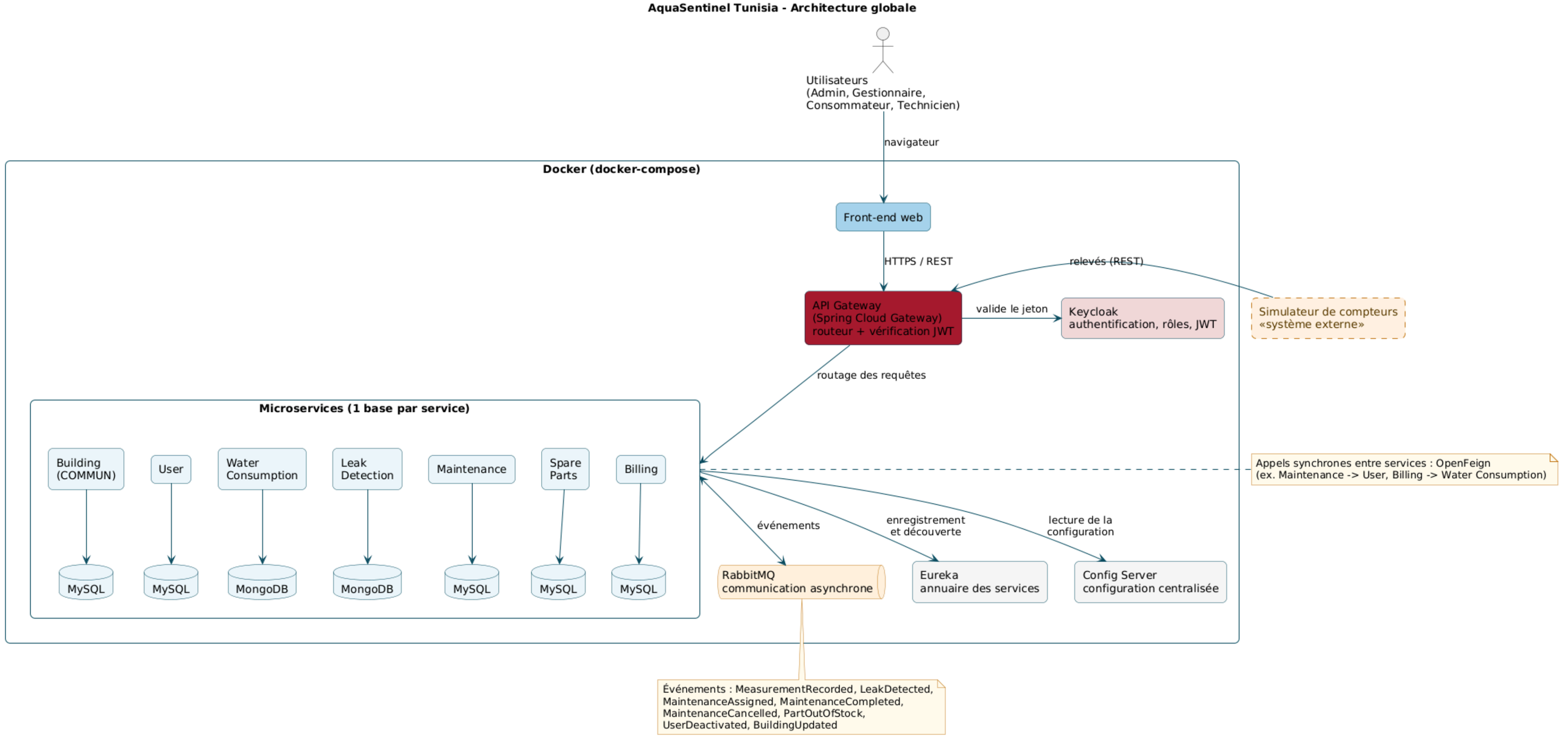
5. STOCK DE PIÈCES

RÉSERVER LES PIÈCES, SIGNALER LES RUPTURES, METTRE À JOUR LE STOCK.

6. FACTURATION ET ÉCONOMIES

CALCULER LA FACTURE PAR TRANCHES ET ESTIMER L'ÉCONOMIE APRÈS RÉPARATION.

L'ARCHITECTURE GLOBALE:



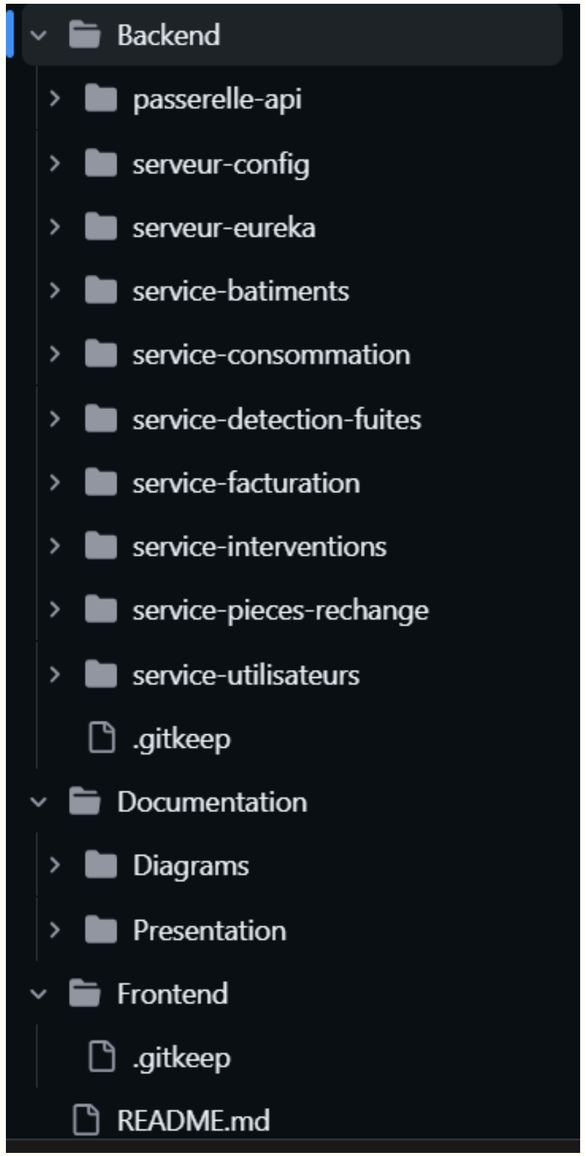
DÉCOUPAGE EN MICROSERVICES

Microservice	Responsabilité	Base	Pourquoi cette base	Responsable
Service des bâtiments (Building Service) – COMMUN	Référentiel des bâtiments, de leur compteur et de leur gestionnaire	MySQL	Données structurées et liées, intégrité des références	Toute l'équipe
Service des utilisateurs (User Service)	Comptes, rôles, authentification et disponibilité des techniciens	MySQL	Données relationnelles, cohérence des comptes	Manar Fares
Service de consommation d'eau (Water Consumption Service)	Enregistrement, historique, agrégation et correction des relevés	PostgreSQL	schéma stable des relevés, contrainte UNIQUE (meterRef, measuredAt), agrégations SQL rapides pour Billing et transactions pour l'outbox.	Yasmine Maatougui
Service de détection des fuites (Leak Detection Service)	Repérage des consommations anormales et évaluation du risque de fuite	MongoDB	Historique volumineux, champs d'analyse variables selon la règle	Kawther Amri
Service des interventions (Maintenance Service)	Organisation et suivi des réparations liées aux fuites	MySQL	Cycle de vie contrôlé (statuts), transactions	Loussaif Bhiri
Service des pièces de rechange (Spare Parts Service)	Stock, réservation et sortie des pièces de réparation	MySQL	Cohérence du stock (réserver, débiter), transactions	Balkiss Youssfi
Service de facturation (Billing Service)	Factures estimées par tranches, majorations et économies	MySQL	Données financières : montants exacts, et une facture définitive ne se modifie plus	Khairi Charrad

LES TECHNOLOGIES CHOISIES:

Domaine	Choix	Rôle dans le projet
Front-end	Angular	Interface web des 4 types d'utilisateurs
Microservices	Java + Spring Boot, Maven	Développement de 6 microservices
Technologie avancée	GraphQL (Spring for GraphQL)	Microservice commun Building, développé par toute l'équipe
API Gateway	Spring Cloud Gateway	Point d'entrée unique, routage, vérification du jeton
Découverte	Netflix Eureka Server	Annuaire des services
Configuration	Spring Cloud Config Server	Configuration centralisée
Sécurité	Keycloak (OAuth2 / OpenID Connect, JWT, rôles, verrouillage après échecs)	Authentification et rôles
Communication synchrone	OpenFeign + Resilience4j (circuit breaker, fallback)	Appels directs entre services, avec gestion des pannes
Communication asynchrone	RabbitMQ (exchange topic aquasentinel.events, dead-letter queue)	Événements entre services
Bases de données	MySQL, MongoDB,PostgreSQL	Une base indépendante par service
Conteneurisation	Docker + Docker Compose	Lancement de tout le système
Documentation API	OpenAPI / Swagger	Contrats d'API figés avant de coder
Gestion de version et de projet	Git + GitHub (branches, pull requests) + GitHub Projects (Kanban)	Versionnement du code, travail en équipe, suivi et répartition des tâches

DÉPÔT GIT ET GESTION DE PROJET:



- **DÉPÔT GIT**
  - BACKEND : UN PROJET PAR MICROSERVICE
  - FRONTEND : ANGULAR
  - DOCUMENTATION : PRÉSENTATION, DIAGRAMMES
- **ORGANISATION**
  - UNE BRANCHE PAR TÂCHE
  - PULL REQUEST AVANT LA FUSION
  - CHAQUE MEMBRE COMMITE SUR SON SERVICE
- **GESTION DE PROJET**
  - GITHUB PROJECTS (KANBAN)

LE PLANNING PRÉVISIONNEL:

Période	Objectif
Semaines 1 à 3	Choix du sujet, découpage en microservices, conception
Semaine 4	Validation 1 : conception, dépôt Git, présentation
Semaines 5 et 6	Infrastructure (Eureka, Config, Gateway, Keycloak, RabbitMQ, Docker) + service commun Building
Semaines 7 à 9	Développement de chaque microservice (CRUD, règles métier, base)
Semaines 10 et 11	Communications OpenFeign et RabbitMQ, simulateur, tests
Semaines 12 et 13	Front-end Angular et intégration
Semaine 14	Tests finaux et validation finale

LES PRINCIPAUX RISQUES :

Risque	Solution prévue
Building est utilisé par tous les services (point de dépendance)	Schéma GraphQL figé ensemble dès le début, message d’erreur clair s’il est indisponible
Un service en panne bloque les autres	Resilience4j (circuit breaker, fallback) et événements RabbitMQ
Message RabbitMQ perdu ou traité deux fois	Dead-letter queue et traitement idempotent des événements
Trop de conteneurs pour un PC	Lancer seulement les services utiles avec Docker Compose
Contrats d’API qui changent en cours de projet	Contrats OpenAPI et GraphQL validés en équipe avant de coder

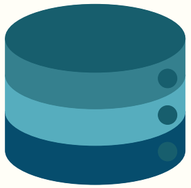
MICRO SERVICE

BUILDING SERVICE (COMMUN)

BUILDING SERVICE : RÔLE MÉTIER & PÉRIMÈTRE

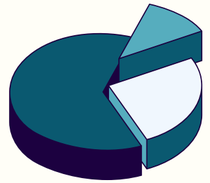
RÔLE UNIQUE & MÉTIER

référentiel unique des bâtiments, de leurs compteurs d'eau et de leur gestionnaire. C'est le service commun : tous les autres services utilisent son identifiant buildingId.



MYSQL (BUILDING_DB)

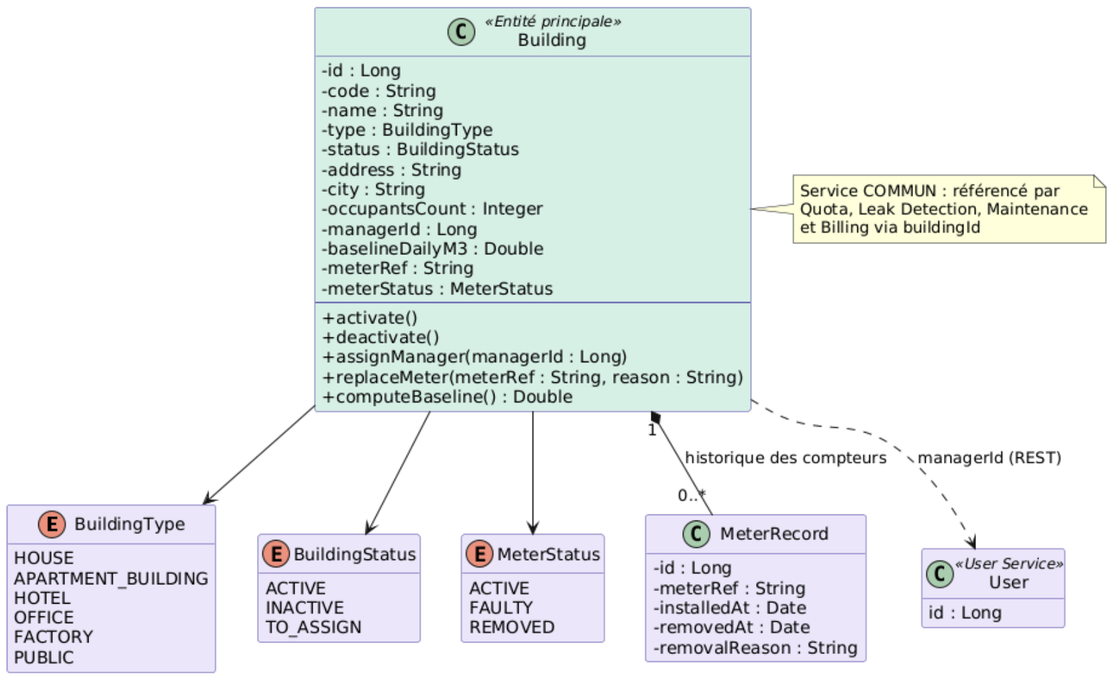
transactions ACID, donc statuts, affectations de gestionnaire et remplacements de compteur cohérents, même en accès simultané.



CONTRIBUTION SERVICE COMMUN

recherche, filtres (ville, type, statut, gestionnaire) et statistiques du parc pour tous les membres.

CONCEPTION : DIAGRAMME DE CLASSES & RÈGLES MÉTIER



- **Entité principale (MaintenanceRequest) :**

Référence un bâtiment surveillé, avec sa localisation, ses caractéristiques, son gestionnaire, son compteur actif et l'historique de ses compteurs.

- **Gestion des états (MaintenanceStatus) :**

Cycle contrôlé : ACTIVE ↔ INACTIVE, avec TO_ASSIGN quand le gestionnaire est désactivé. Un bâtiment repasse à ACTIVE dès qu'un nouveau gestionnaire lui est affecté. Un bâtiment INACTIVE n'accepte plus de nouveau compteur ni de nouveau gestionnaire.

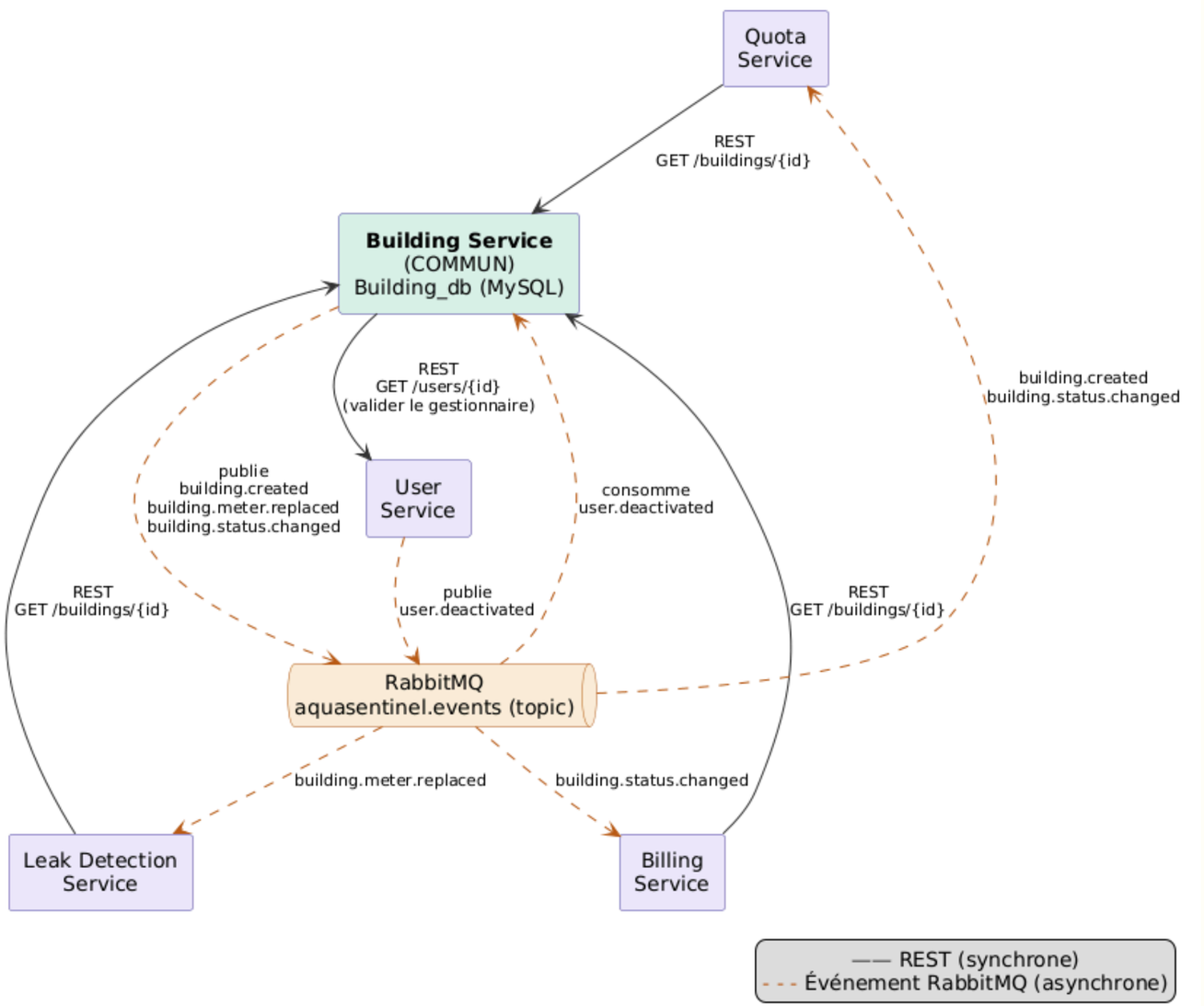
- **Type (BuildingType) :**

Maison, immeuble, hôtel, bureau, usine ou bâtiment public. Il sert à calculer la consommation de référence et à déterminer la catégorie de facturation.

- **Méthodes clés :**

Pilotage du cycle de vie et du compteur via activate(), deactivate(), assignManager(), replaceMeter() et computeBaseline().

INTERACTIONS DU BUILDING SERVICE



• Communications synchrones (REST / OpenFeign) :

→ User : GET /api/users/{id}, pour valider le gestionnaire avant de lui affecter un bâtiment.

→ Appelé par Leak Detection et Billing (GET /api/buildings/{id}), qui ont besoin des informations du bâtiment et de son compteur.

• Communications asynchrones (RabbitMQ) :

→ Publie building.created, building.meter.replaced et building.status.changed. Les autres services sont informés d'un nouveau bâtiment, d'un changement de compteur ou de statut.

→ Consomme user.deactivated (de User) : le bâtiment passe à TO_ASSIGN quand son gestionnaire est désactivé.

• Tolérance aux pannes :

Si User est indisponible : réponse 503 et nouvelle tentative (retry). Aucun bâtiment n'est créé avec un gestionnaire non vérifié.

CONTRATS D'API ENDPOINTS & CODES DE RÉPONSE

REST

POST	/api/buildings	Création d'un bâtiment (gestionnaire vérifié)	201 / 400 / 409 / 503
GET	/api/buildings	Liste et filtres (ville, type, statut)	200
GET	/api/buildings/{id}	Consultation d'un bâtiment	200 / 404
PUT	/api/buildings/{id}	Modification d'un bâtiment	200 / 400 / 404
PUT	/api/buildings/{id}/meter	Remplacement du compteur	200 / 404 / 409
PATCH	/api/buildings/{id}/manager	Affectation d'un gestionnaire	200 / 404 / 409
DELETE	/api/buildings/{id}	Suppression si aucune donnée liée	204 / 404 / 409

GraphQL (technologie avancée)

QUERY	building(id)	Bâtiment + compteur + gestionnaire en 1 requête	200 / errors
QUERY	buildings(city, type, status)	Recherche avec champs choisis	200
QUERY	building(id){ baselineDailyM3 }	Consommation de référence (Leak Detection, Billing)	200 / errors

- REST : CRÉER, LISTER, CONSULTER, MODIFIER ET SUPPRIMER UN BÂTIMENT, REMPLACER SON COMPTEUR, AFFECTER UN GESTIONNAIRE.
- GRAPHQL (TECHNOLOGIE AVANCÉE) : CHAQUE SERVICE DEMANDE SEULEMENT LES CHAMPS UTILES, EN UNE SEULE REQUÊTE.
- 409 : ACTION IMPOSSIBLE, PAR EXEMPLE SUPPRIMER UN BÂTIMENT QUI A DÉJÀ DES RELEVÉS, OU MODIFIER UN BÂTIMENT INACTIVE.
- 503 : USER INDISPONIBLE, DONC LE GESTIONNAIRE NE PEUT PAS ÊTRE VÉRIFIÉ ET LA CRÉATION EST REFUSÉE.

CONTRATS D'ÉVÉNEMENTS & GESTION DES ERREURS

↑ BuildingUpdated v1 — publié → Water Consumption, Leak Detection, Billing

eventId	UUID	idempotence
buildingId	Long	bâtiment concerné
changeType	Enum	CREATED / METER_REPLACED / STATUS_CHANGED
meterRef	String	compteur actif
status	Enum	ACTIVE / INACTIVE / TO_ASSIGN

↓ UserDeactivated v1 — consommé ← User

eventId	UUID	idempotence
userId	Long	gestionnaire désactivé
role	Enum	MANAGER

Effet : ses bâtiments passent à TO_ASSIGN.

Format d'erreur commun (exemple : 503)

```
{ "status": 503, "error": "Service Unavailable",
  "message": "User Service indisponible : gestionnaire non vérifié",
  "path": "/api/buildings" }
```

- **PUBLIE BUILDINGUPDATED :** PRÉVIENT WATER CONSUMPTION, LEAK DETECTION ET BILLING D'UNE CRÉATION, D'UN CHANGEMENT DE COMPTEUR OU DE STATUT.
- **CONSOMME USERDEACTIVATED :** LES BÂTIMENTS DU GESTIONNAIRE DÉSACTIVÉ PASSENT À TO_ASSIGN.
- **IDEMPOTENCE :** CHAQUE ÉVÉNEMENT A UN EVENTID UNIQUE, IL N'Y A DONC AUCUN DOUBLON.
- **RÉSILIENCE :** DÉLAI MAXIMAL ET NOUVELLE TENTATIVE VERS USER, PUIS DEAD-LETTER QUEUE APRÈS PLUSIEURS ÉCHECS.
- **ERREURS :** MÊME FORMAT QUE DANS TOUS LES SERVICES.

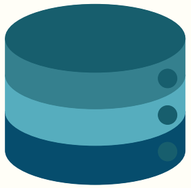
MICRO SERVICE

SPARE PARTS SERVICE

SPARE PARTS SERVICE : RÔLE MÉTIER & PÉRIMÈTRE

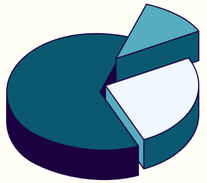
RÔLE UNIQUE & MÉTIER

Garantir la disponibilité opérationnelle des pièces de rechange avant le déplacement des techniciens. Gestion du stock physique et virtuel.



MYSQL (PARTS_DB)

Garantie des transactions ACID lors des réservations simultanées de pièces par plusieurs demandes d'intervention concurrentes.



CONTRIBUTION SERVICE COMMUN

Développement du module d'importation CSV massif des bâtiments et compteurs pour le Building Service (COMMUN).

INTERACTIONS SYNCHRONES (REST) & ASYNCHRONES (RABBITMQ)

Appels Synchrones (OpenFeign)

- **Maintenance → Spare Parts**

Vérification et réservation synchrone de la pièce lors de la création d'intervention. Blocage immédiat des pièces.

Événements RabbitMQ

- ***Publie*** **PartOutOfStock**

Émis si stock insuffisant → Intervention bascule au statut "En attente de pièce".

- ***Consomme*** **MaintenanceCompleted / Cancelled**

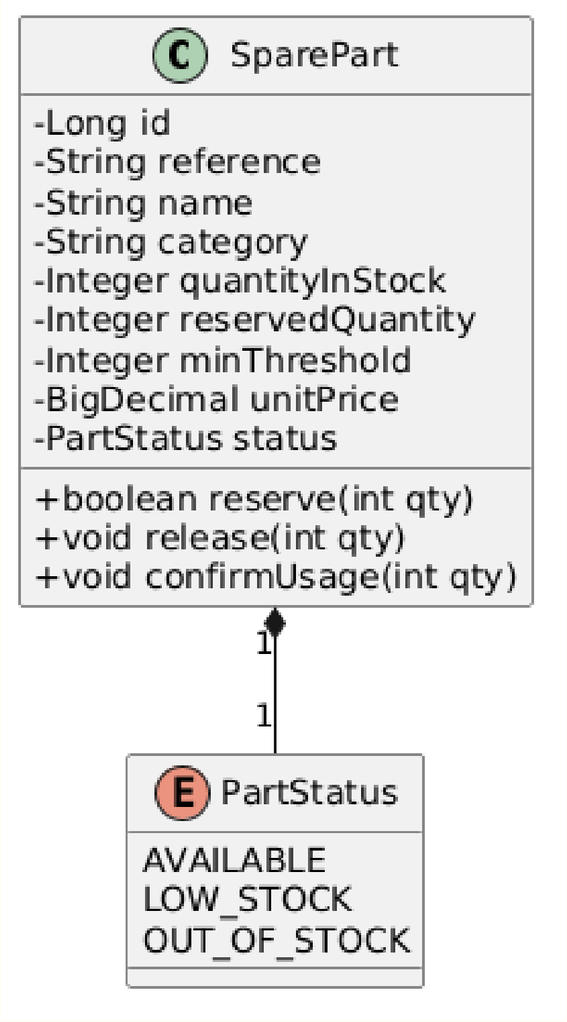
Décrémentation définitive du stock ou libération des pièces réservées.

[Maintenance Service] -- (REST OpenFeign: POST /api/parts/reserve) --> [Spare Parts Service]
[Spare Parts Service] -- (RabbitMQ Event: PartOutOfStock) ----------> [Maintenance Service]
[Maintenance Service] -- (RabbitMQ Event: MaintenanceCompleted) ------> [Spare Parts Service (Décrémente Stock)]

CONTRATS D'API ENDPOINTS & CODES DE RÉPONSE

GET	/api/parts	Recherche et filtrage des pièces (catégorie, statut, référence)	200 OK
POST	/api/parts	Création d'une référence de pièce de rechange	201 Created
POST	/api/parts/reserve	Réservation de pièces pour une demande d'intervention	200 OK / 400 OutOfStock
POST	/api/parts/release	Libération des pièces réservées (annulation intervention)	200 OK
PUT	/api/parts/{id}/restock	Réapprovisionnement du stock et mise à jour du statut	200 OK / 404 Not Found

CONCEPTION : DIAGRAMME DE CLASSES & RÈGLES MÉTIER



- **Entité principale (SparePart) :**

Gère l'inventaire en temps réel, les seuils d'alerte et la valorisation du stock.

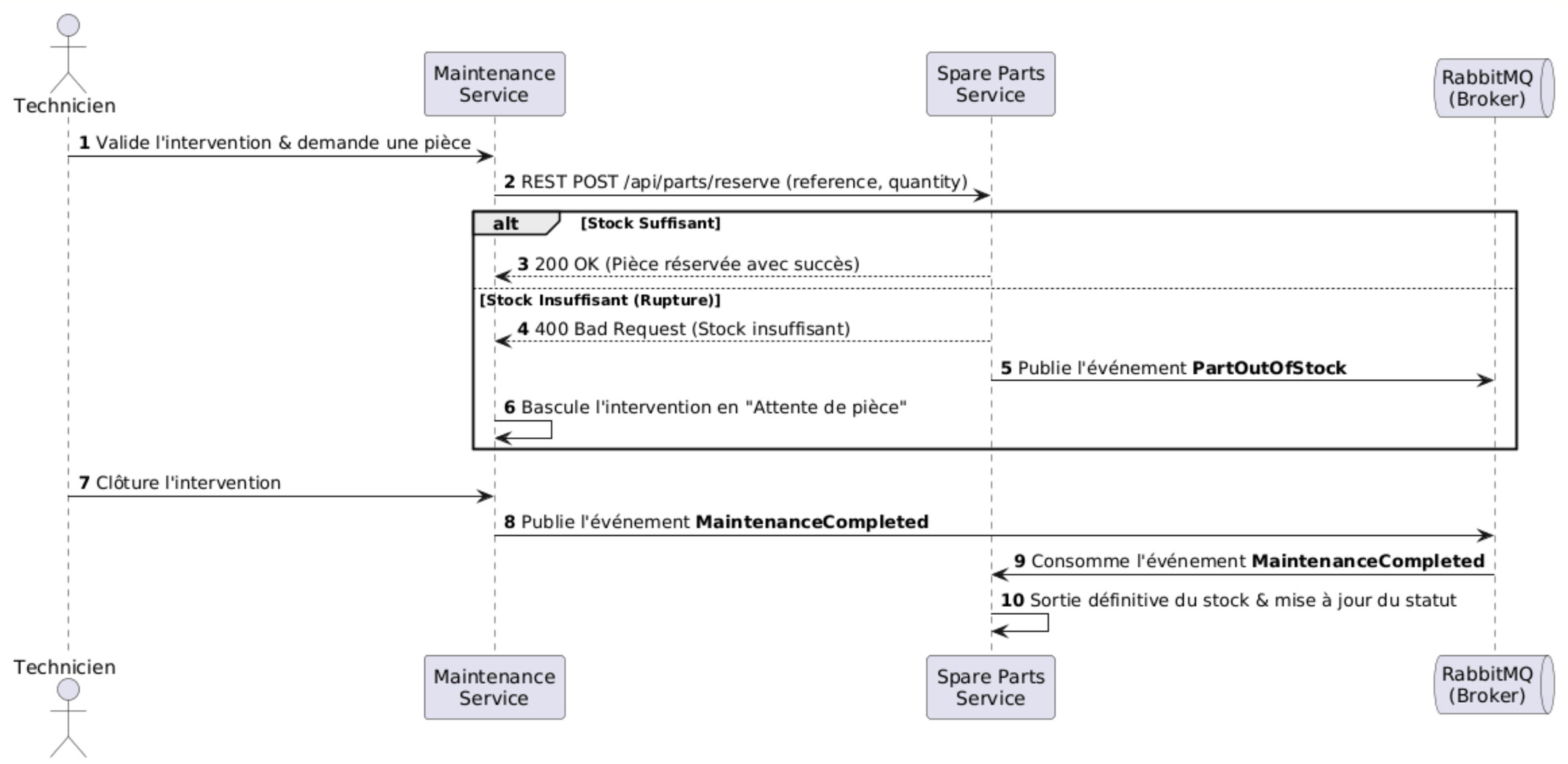
- **Gestion des états (PartStatus) :**

Bascule automatique entre AVAILABLE (disponible), LOW_STOCK (stock faible) et OUT_OF_STOCK (rupture).

- **Méthodes clés :**

Sécurisation des stocks via la réservation (reserve()), la libération (release()) et la sortie définitive (confirmUsage()).

CONCEPTION : INTERACTIONS & DIAGRAMME DE SÉQUENCE

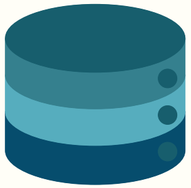


MICRO SERVICE
MAINTENANCE SERVICE

MAINTENANCE SERVICE : RÔLE MÉTIER & PÉRIMÈTRE

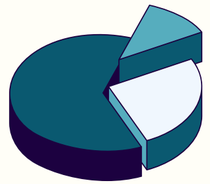
RÔLE UNIQUE & MÉTIER

Transformer une fuite détectée en intervention suivie jusqu'à la clôture : affectation du technicien, réservation des pièces et calcul du coût final.



MYSQL (MAINTENANCE_DB)

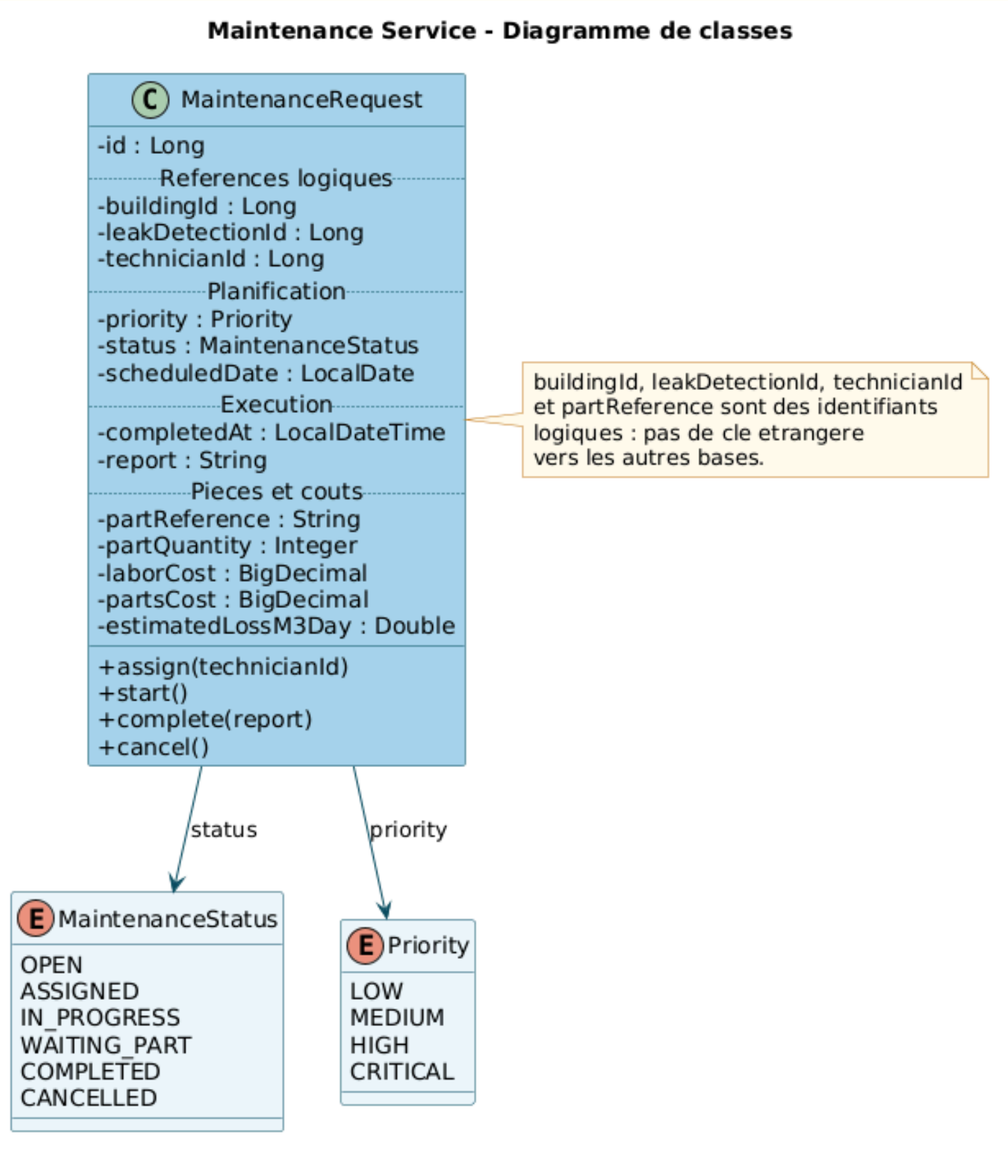
Garantie des transactions ACID : les changements de statut (ouverte, affectée, en cours, terminée) et les coûts restent cohérents, même avec plusieurs interventions simultanées.



CONTRIBUTION SERVICE COMMUN

Développement de la recherche et des filtres (ville, type, statut, gestionnaire) ainsi que des statistiques du parc pour le Building Service (COMMUN).

CONCEPTION : DIAGRAMME DE CLASSES & RÈGLES MÉTIER



- **Entité principale (MaintenanceRequest) :**

Suit une intervention technique de la fuite détectée jusqu'à la clôture, avec le technicien affecté, la pièce réservée et le coût final.

- **Gestion des états (MaintenanceStatus) :**

Cycle contrôlé : OPEN → ASSIGNED → IN_PROGRESS → COMPLETED, avec WAITING_PART en cas de rupture de pièce et CANCELLED avant le démarrage.

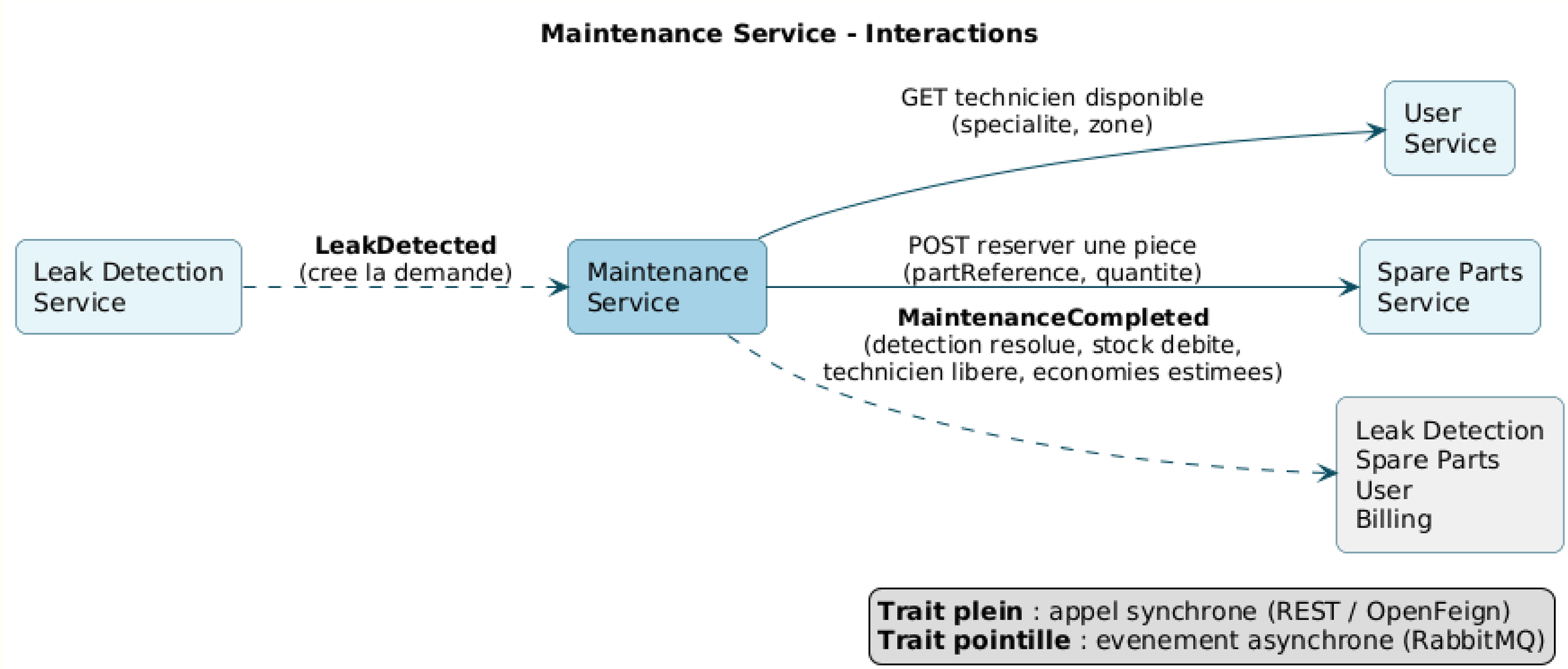
- **Priorité (Priority) :**

De LOW à CRITICAL, déduite du niveau de risque de la fuite.

- **Méthodes clés :**

Pilotage du cycle de vie via assign(), start(), complete() et cancel().

INTERACTIONS DU MAINTENANCE SERVICE



CONTRATS D'API ENDPOINTS & CODES DE RÉPONSE

GET	`/api/maintenance-requests`	Liste et filtrage (statut, priorité, technicien, bâtiment)	`200 OK`
POST	`/api/maintenance-requests/{id}/assign`	Affectation d'un technicien	`200 OK / 409 Conflict / 503 Unavailable`
POST	`/api/maintenance-requests/{id}/start`	Démarrage de l'intervention	`200 OK / 409 Conflict`
POST	`/api/maintenance-requests/{id}/complete`	Clôture : compte rendu et coût final	`200 OK / 409 Conflict`
POST	`/api/maintenance-requests/{id}/cancel`	Annulation (intervention non commencée)	`200 OK / 409 Conflict`

CONTRATS D'ÉVÉNEMENTS & GESTION DES ERREURS

CONSUME **LeakDetected**

`leakDetectionId · buildingId · riskLevel · estimatedLossM3Day · detectedAt`

CONSUME **UserDeactivated**

`userId · role`

PUBLISH **MaintenanceCompleted**

`maintenanceId · buildingId · leakDetectionId · technicianId · partReference · partQuantity · estimatedLossM3Day · completedAt`

REGLES DU CONTRAT:

- · Événements versionnés, contenu minimal
- Pas de doublon : une fuite = une seule demande
- . Message rejoué toléré

User indisponible
Timeout : la demande reste OPEN, l'affectation est retentée (503 en appel manuel)

Spare Parts indisponible
Réservation retentée : l'intervention ne démarre pas sans pièce

Rupture de pièce
Réponse 400 OutOfStock : statut WAITING_PART

Transition interdite
Réponse 409 Conflict : le statut n'est pas modifié

Message en double
Ignoré grâce à l'idempotence (une fuite = une seule demande)

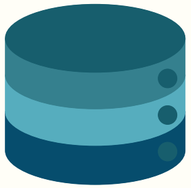
Échec de traitement d'un message
Nouvelles tentatives, puis file d'erreurs (dead-letter queue)

MICRO SERVICE
USER SERVICE

USER SERVICE: RÔLE MÉTIER & PÉRIMÈTRE

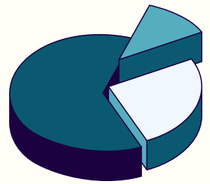
RÔLE UNIQUE & MÉTIER

Savoir qui peut faire quoi : authentifier les utilisateurs, contrôler leurs droits par rôle et connaître les techniciens disponibles pour intervenir.



MONGODB (PARTS_DB)

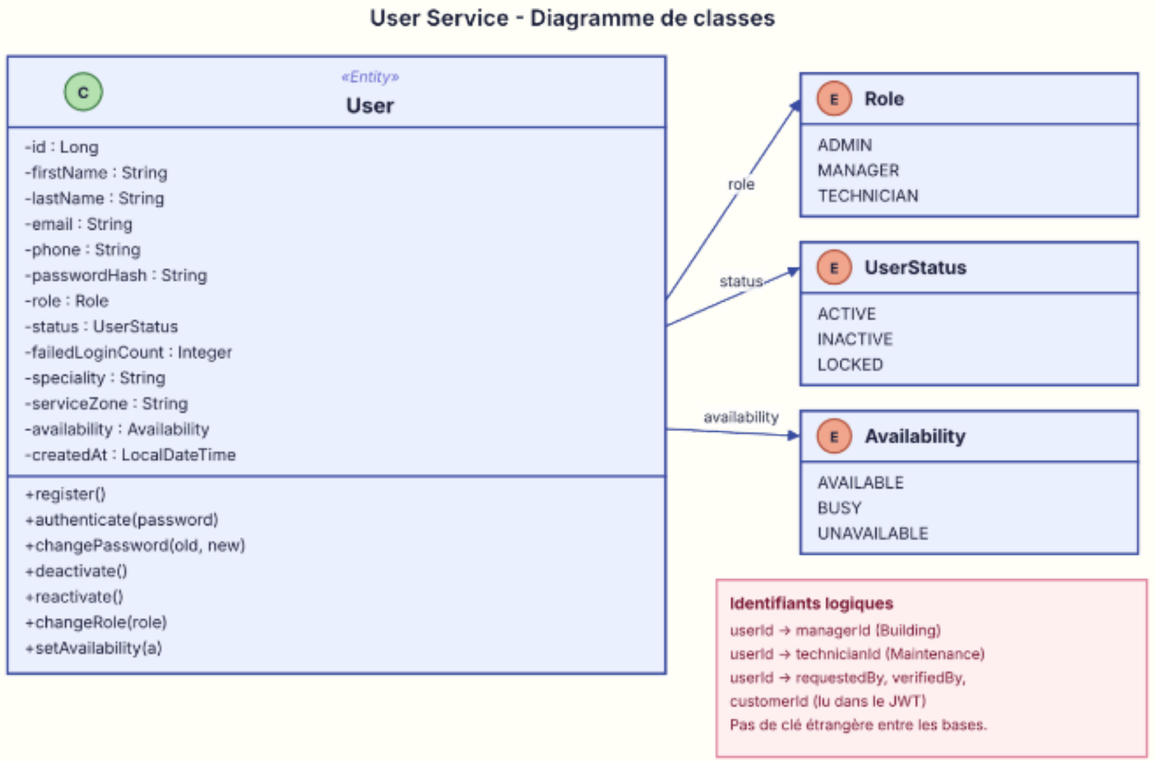
Base indépendante, sans clé étrangère : comptes, rôles, infos techniciens (spécialité, zone, disponibilité) et compteur d'échecs de connexion.



CONTRIBUTION SERVICE COMMUN

Valide le gestionnaire (managerId) d'un bâtiment, appelé en REST par Building. Publie UserDeactivated : le bâtiment passe à « à affecter ».

CONCEPTION : DIAGRAMME DE CLASSES & RÈGLES MÉTIER



Entité principale (User) : compte de la plateforme : identité, sécurité et, pour les techniciens, spécialité, zone et disponibilité.

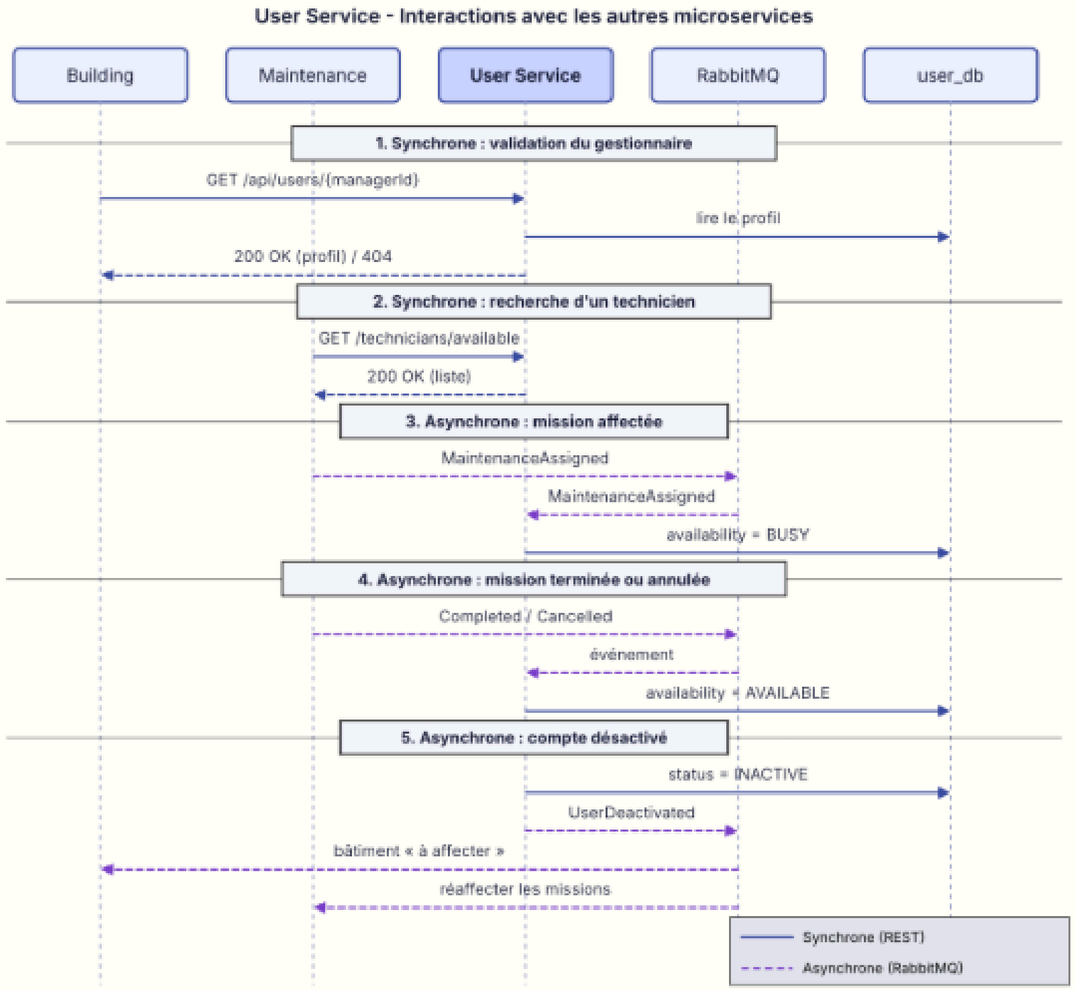
Rôles (Role) : ADMIN, MANAGER, TECHNICIAN. Le contrôle d'accès (RBAC) passe par l'API Gateway.

Statut (UserStatus) : ACTIVE → LOCKED après plusieurs échecs de connexion ; INACTIVE si désactivé.

Disponibilité : AVAILABLE ↔ BUSY selon les missions de Maintenance.

Règles clés : email unique, mot de passe haché, rôle modifiable par l'administrateur seulement.

LES INTERACTIONS AVEC LES AUTRES MICRO SERVICES:



Communications synchrones (REST)

Building → User : GET /api/users/{id}, validation du gestionnaire
Maintenance → User : GET /api/users/technicians/available

Communications asynchrones (RabbitMQ)

Publie UserDeactivated (vers Building et Maintenance)
Reçoit MaintenanceAssigned : technicien BUSY
Reçoit MaintenanceCompleted / MaintenanceCancelled : technicien AVAILABLE

Tolérance aux pannes

Réessais (retry), puis dead-letter queue pour les échecs persistants
eventId mémorisé : traitement idempotent des événements

CONTRATS D'API ENDPOINTS & CODES DE RÉPONSE

POST	/api/auth/register	Inscription	201 / 400 / 409
POST	/api/auth/login	Connexion, renvoie un JWT	200 / 401 / 423
POST	/api/auth/refresh	Renouvellement du jeton	200 / 401
GET	/api/users	Liste et filtres (rôle, statut, ville)	200
GET	/api/users/{id}	Consultation d'un profil	200 / 404
PUT	/api/users/{id}	Modification du profil	200 / 400 / 404
PATCH	/api/users/{id}/password	Changer le mot de passe	200 / 400 / 401 / 404
PATCH	/api/users/{id}/deactivate	Désactiver un compte	200 / 404 / 409
PATCH	/api/users/{id}/reactivate	Réactiver un compte	200 / 404 / 409
PATCH	/api/users/{id}/role	Changer le rôle (admin)	200 / 403 / 404
GET	/api/users/technicians/available	Techniciens disponibles	200

- GET : LISTER ET CONSULTER
- POST : S'INSCRIRE, SE CONNECTER
- PATCH : MOT DE PASSE, RÔLE, ÉTAT DU COMPTE
- 409 : ACTION IMPOSSIBLE · 423 : COMPTE VERROUILLÉ

CONTRATS D'ÉVÉNEMENTS & GESTION DES ERREURS

↑ UserDeactivated v1 : publié → Building, Maintenance

eventId	UUID	idempotence
userId	Long	utilisateur désactivé
role	Enum	ADMIN / MANAGER / TECHNICIAN
deactivatedAt	DateTime	date de désactivation

↓ MaintenanceAssigned v1 : consommé ← Maintenance

eventId	UUID	idempotence
maintenanceId	Long	intervention
technicianId	Long	technicien → BUSY

↓ MaintenanceCompleted / MaintenanceCancelled v1 ← Maintenance

eventId	UUID	idempotence
maintenanceId	Long	intervention
technicianId	Long	technicien → AVAILABLE

Format d'erreur commun (exemple : 423)

```
{ "status": 423, "error": "Locked",
  "message": "Compte verrouillé : trop d'échecs",
  "path": "/api/auth/login" }
```

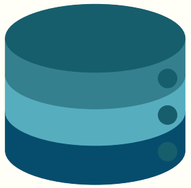
- Publie UserDeactivated → Building passe à « à affecter », Maintenance réaffecte les missions.
- Consomme MaintenanceAssigned → technicien BUSY.
- Consomme MaintenanceCompleted / MaintenanceCancelled → technicien AVAILABLE.
- Idempotence : eventId unique, aucun doublon.
- Résilience : retry, puis dead-letter queue, aucun message perdu.
- Erreurs : format JSON commun à tous les services.
-

MICRO SERVICE
LEAK DETECTION SERVICE

LEAK DETECTION SERVICE : RÔLE MÉTIER & PÉRIMÈTRE

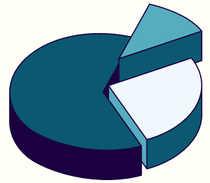
RÔLE UNIQUE & MÉTIER

Transformer les relevés de consommation en alertes exploitables : détecter les fuites, évaluer leur niveau de risque et estimer la perte d'eau, avant la réception de la facture.



MONGODB (PARTS_DB)

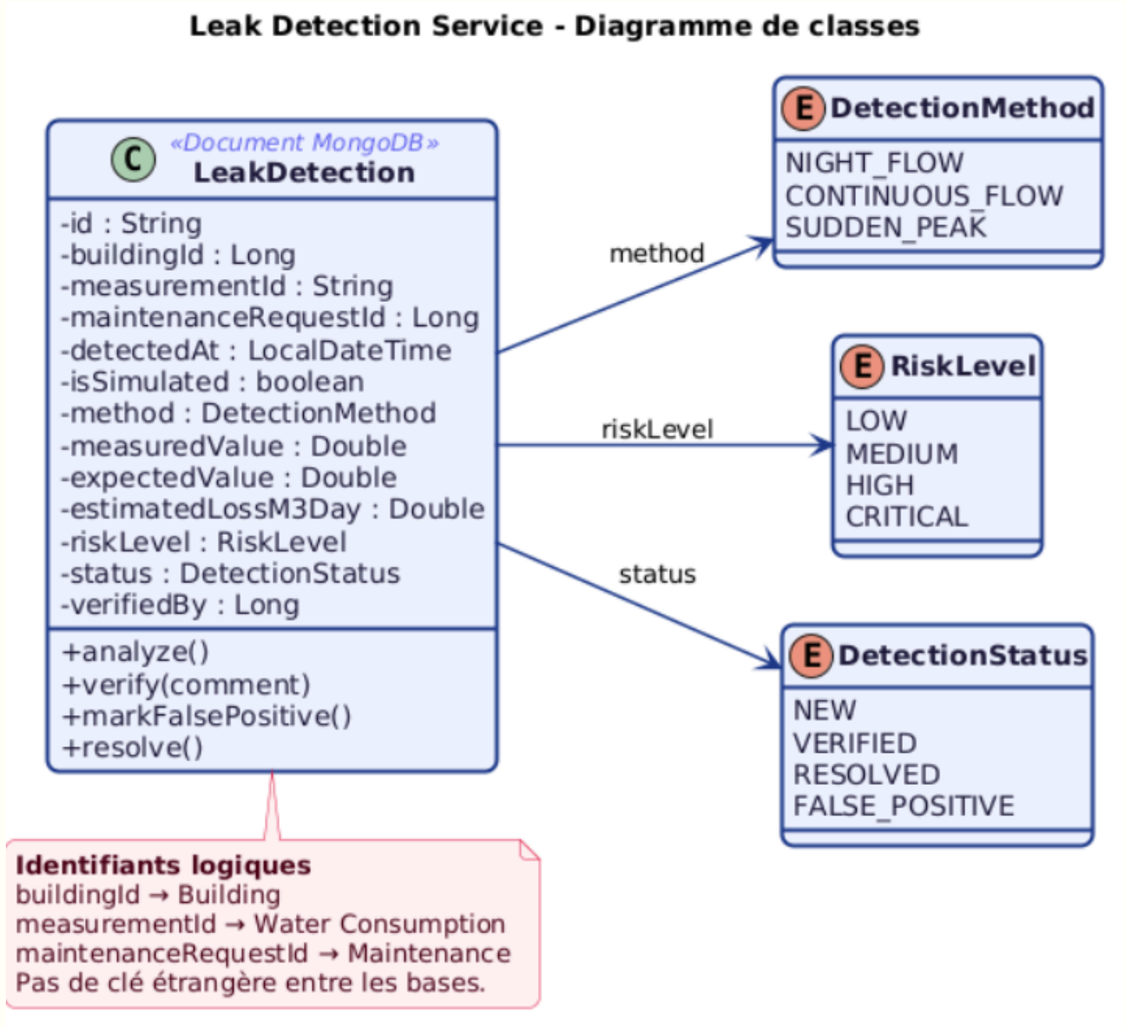
Stockage flexible des détections : les champs d'analyse varient selon la règle appliquée (débit nocturne, débit continu, pic soudain), avec un historique volumineux.



CONTRIBUTION SERVICE COMMUN

Développement du calcul de la consommation journalière de référence (baselineDailyM3) des bâtiments, exposée en GraphQL par le Building Service (COMMUN).

CONCEPTION : DIAGRAMME DE CLASSES & RÈGLES MÉTIER



• Entité principale (LeakDetection) :
Enregistre chaque fuite potentielle détectée à partir d'un relevé, avec la valeur mesurée, la valeur attendue et la perte d'eau estimée en m³ par jour.

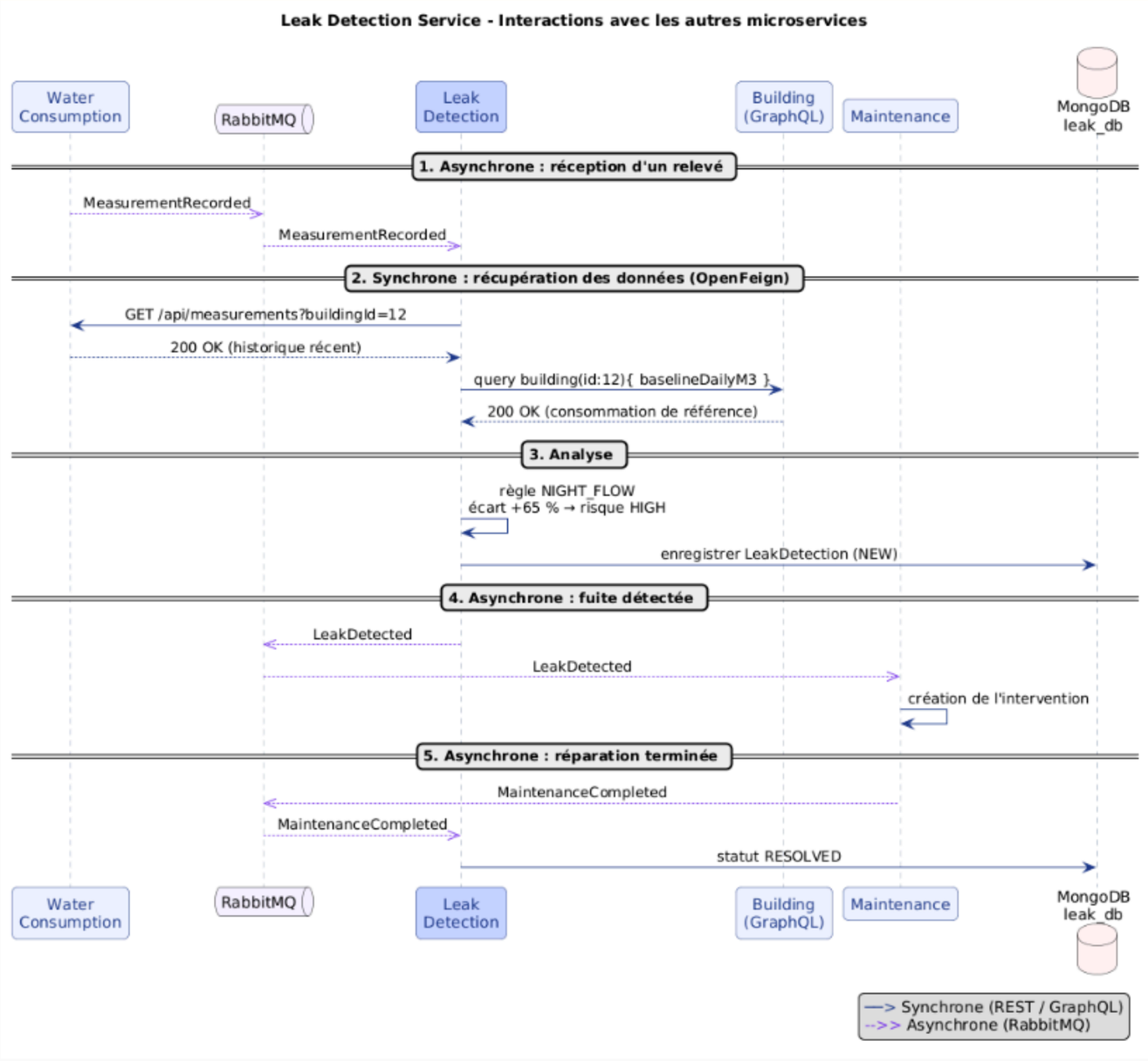
• Méthodes de détection (DetectionMethod) :
Trois règles métier : NIGHT_FLOW (débit anormal la nuit), CONTINUOUS_FLOW (débit qui ne s'arrête jamais) et SUDDEN_PEAK (pic soudain).

• Niveau de risque (RiskLevel) :
De LOW à CRITICAL, calculé selon l'écart entre la consommation mesurée et la consommation attendue. Il fixe la priorité de l'intervention.

• Gestion des états (DetectionStatus) :
Cycle contrôlé : NEW → VERIFIED → RESOLVED, avec FALSE_POSITIVE en cas de fausse alerte.

• Méthodes clés :
Pilotage du cycle de vie via analyze(), verify(), markFalsePositive() et resolve().

LES INTERACTIONS AVEC LES AUTRES MICRO SERVICES:



Communications synchrones (OpenFeign / GraphQL)

→ Water Consumption : GET /api/measurements, pour l'historique récent des relevés

→ Building (GraphQL) : baselineDailyM3, pour la consommation de référence

Communications asynchrones (RabbitMQ)

Reçoit MeasurementRecorded (de Water Consumption) : démarre l'analyse

Publie LeakDetected (vers Maintenance) : création de l'intervention

Reçoit MaintenanceCompleted (de Maintenance) : la fuite passe à RESOLVED

Tolérance aux pannes

Message rejoué (retry), puis placé en dead-letter queue : aucune analyse n'est perdue

measurementId mémorisé : un relevé n'est jamais analysé deux fois

CONTRATS D'API ENDPOINTS & CODES DE RÉPONSE

GET	/api/leaks	Liste et filtrage (bâtiment, risque, statut)	200
GET	/api/leaks/{id}	Consultation d'une détection	200 / 404
POST	/api/leaks/simulate	Simulation d'une fuite (démonstration)	201 / 400
PATCH	/api/leaks/{id}/verify	Confirmation par le gestionnaire	200 / 404 / 409
PATCH	/api/leaks/{id}/false-positive	Marquage comme fausse alerte	200 / 404 / 409
DELETE	/api/leaks/{id}	Suppression d'une détection de test	204 / 404

- GET : LISTER ET CONSULTER LES FUITES
- POST : SIMULER UNE FUITE POUR LA DÉMONSTRATION
- PATCH : VÉRIFIER UNE FUITE OU LA MARQUER COMME FAUSSE ALERTE
- 409 : ACTION IMPOSSIBLE SELON LE STATUT (FUITE DÉJÀ RESOLVED)

CONTRATS D'ÉVÉNEMENTS & GESTION DES ERREURS

↑ UserDeactivated v1 : publié → Building, Maintenance

eventId	UUID	idempotence
userId	Long	utilisateur désactivé
role	Enum	ADMIN / MANAGER / TECH.
deactivatedAt	DateTime	date de désactivation

↓ MaintenanceAssigned v1 : consommé ← Maintenance

eventId	UUID	idempotence
maintenanceId	Long	intervention
technicianId	Long	technicien → BUSY

↓ MaintenanceCompleted / Cancelled v1 ← Maintenance

eventId	UUID	idempotence
maintenanceId	Long	intervention
technicianId	Long	technicien → AVAILABLE

Format d'erreur commun (exemple : 423)

```
{ "status": 423, "error": "Locked",
  "message": "Compte verrouillé : trop d'échecs",
  "path": "/api/auth/login" }
```

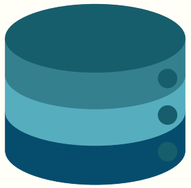
- **Publie LeakDetected → Maintenance crée l'intervention**
- **Consomme MeasurementRecorded → lance l'analyse**
- **Consomme MaintenanceCompleted → fuite RESOLVED**
- **Idempotence : eventId unique, aucun doublon**
- **Résilience : retry, puis dead-letter queue, aucun message perdu**
- **Erreurs : format commun à tous les services**

MICRO SERVICE
BILLING SERVICE

BILLING SERVICE : RÔLE MÉTIER & PÉRIMÈTRE

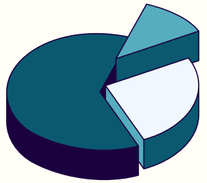
RÔLE UNIQUE & MÉTIER

Chiffrer la consommation d'eau : calculer la facture par tranches tarifaires progressives, appliquer la majoration au-delà du quota et estimer les économies après réparation d'une fuite.



MYSQL (BILLING_DB)

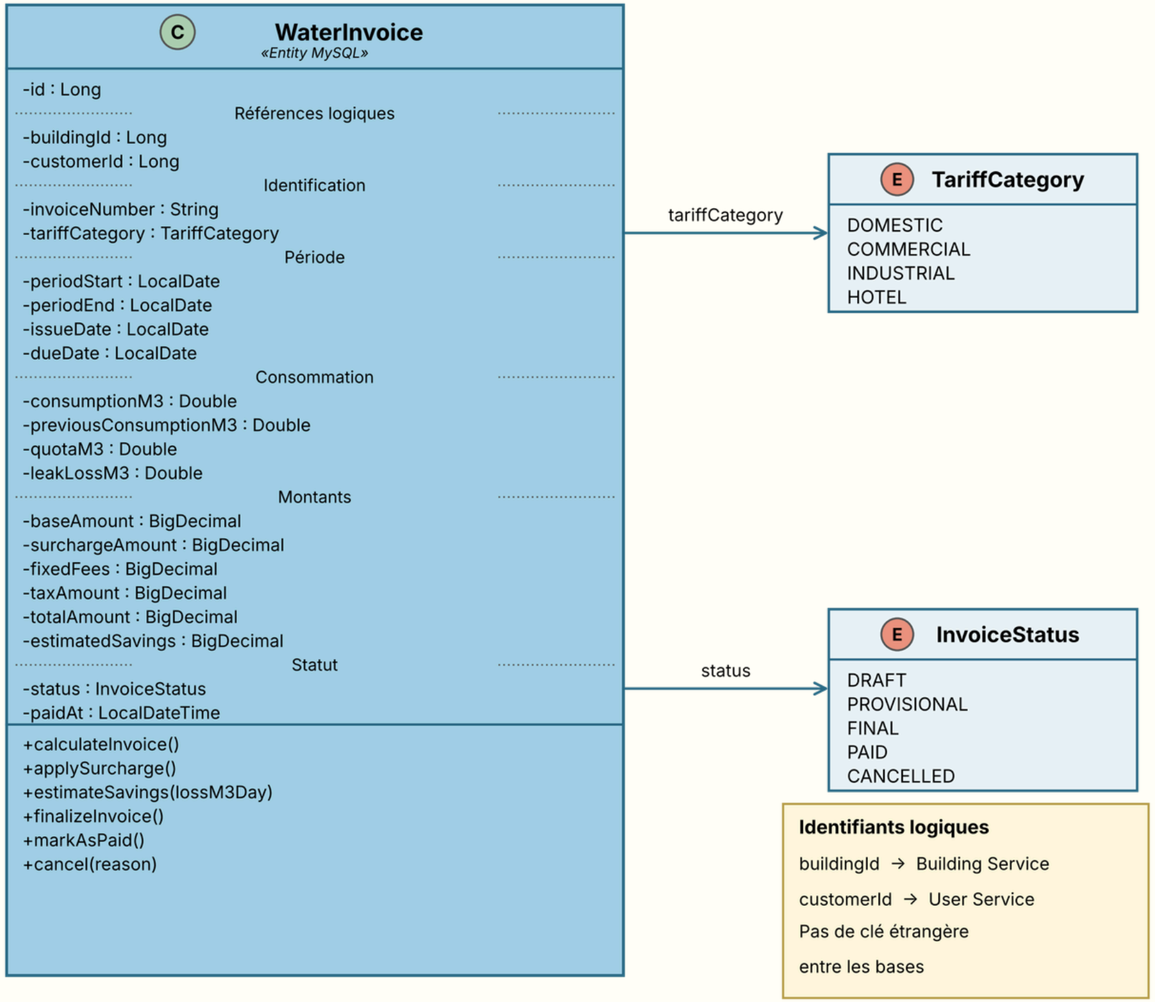
Données financières : montants exacts (BigDecimal), transactions ACID, et une facture définitive ne se modifie plus.



CONTRIBUTION SERVICE COMMUN

Développement de la catégorie tarifaire des bâtiments et sécurisation des appels du Billing Service vers le Building Service (COMMUN).

CONCEPTION : DIAGRAMME DE CLASSES & RÈGLES MÉTIER



Entité principale (WaterInvoice) : : Enregistre les factures de consommation d'eau, avec la période, les volumes consommés en m³, les montants ventilés et les économies estimées après la réparation d'une fuite.

- Catégories tarifaires (TariffCategory) :

Quatre barèmes selon l'usage du bâtiment : DOMESTIC, COMMERCIAL, INDUSTRIAL et HOTEL.

- Gestion des états (InvoiceStatus) :

Cycle contrôlé : Cycle contrôlé : DRAFT → PROVISIONAL → FINAL (facture définitive, immuable) → PAID, avec CANCELLED en cas d'annulation..

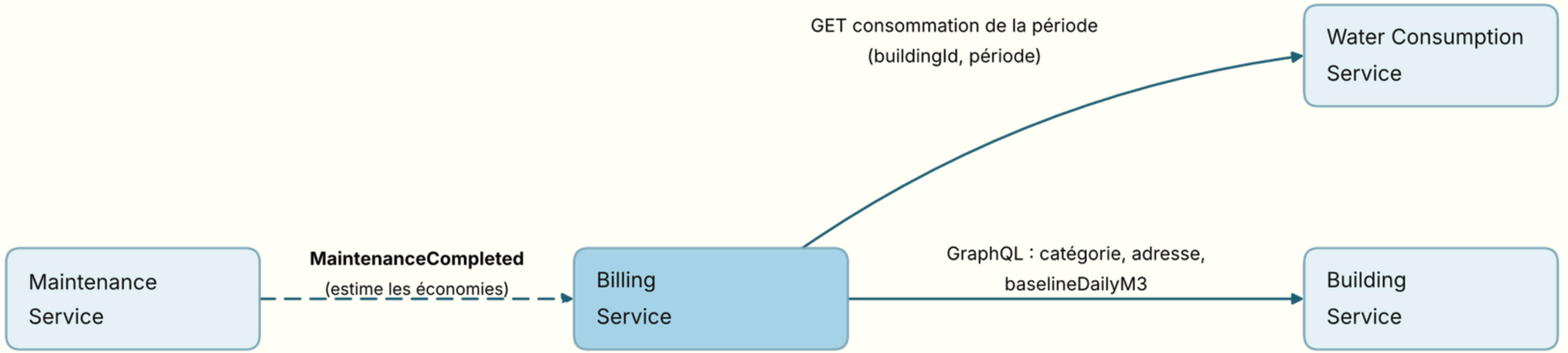
- Règles métier & Calculs :

Calcul par tranches tarifaires progressives, majoration au-delà du quota (quota = consommation de référence du bâtiment × nombre de jours).

- Méthodes clés :

calculateInvoice(), applySurcharge(), estimateSavings(), finalizeInvoice(), markAsPaid() et cancel().

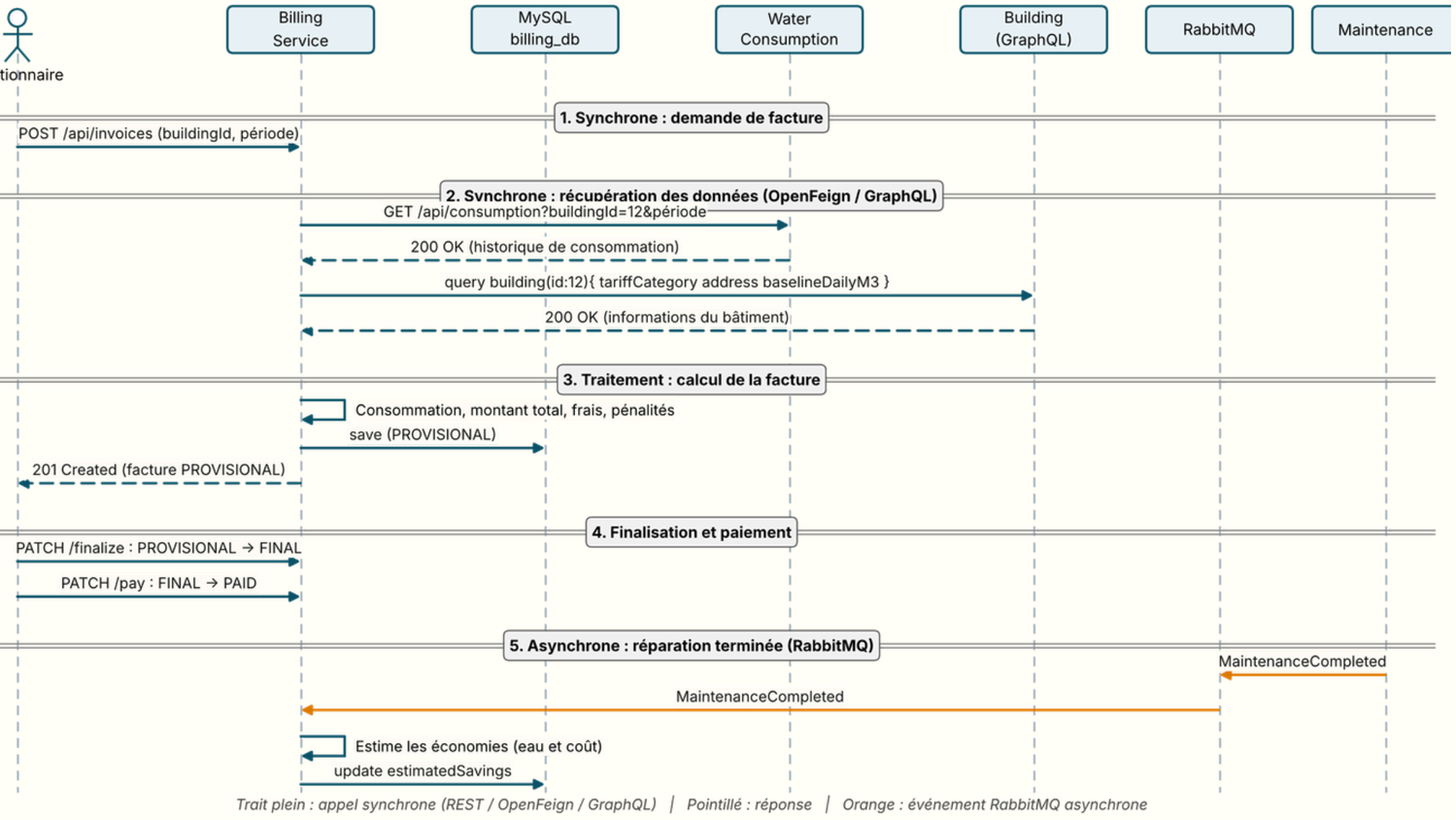
INTERACTIONS DU BILLING SERVER



Resilience4j : circuit breaker et fallback si Water Consumption ou Building est indisponible

Trait plein : appel synchrone (REST / OpenFeign / GraphQL)
Trait pointillé : événement asynchrone (RabbitMQ)

LES INTERACTIONS AVEC LES AUTRES MICRO SERVICES:



Communications synchrones (OpenFeign / GraphQL)

→ Water Consumption : GET /api/consumption, pour récupé
la consommation du bâtiment pendant la période de factur
→ Building (GraphQL) : tariffCategory, address et
baselineDailyM3, pour obtenir les informations du bâtime
calculer la facture

Communications asynchrones (RabbitMQ)

RReçoit MaintenanceCompleted (de Maintenance) : confirme
réparation d'une fuite
→ Estime les économies d'eau et financières réalisées
(perte évitée × tarif)
→ Met à jour estimatedSavings dans la facture de la pé
concernée (DRAFT ou PROVISIONAL)

Traitement des factures

Calcul du montant : tranches tarifaires progressives,
majoration au-delà du quota, frais fixes et taxes
Cycle : DRAFT → PROVISIONAL → FINAL (non modifiable), p
PAID après paiement

Tolérance aux pannes

Water Consumption ou Building indisponible : Resilience4
(circuit breaker, fallback), la génération renvoie 503
Message rejoué (retry), puis dead-letter queue : aucune
estimation d'économies n'est perdue
Traitement idempotent de l'événement : une même réparat
ne modifie jamais deux fois la facre

CONTRATS D'API ENDPOINTS & CODES DE RÉPONSE:

GET	/api/invoices	Liste et filtrage des factures (bâtiment, période, statut)	200 OK
GET	/api/invoices/{id}	Consultation d'une facture	200 OK / 404 Not Found
POST	/api/invoices	Génération d'une facture brouillon	201 Created / 400 Bad Request / 503 Unavailable
DELETE	/api/invoices/{id}	Suppression d'un brouillon	204 No Content / 409 Conflict
PATCH	/api/invoices/{id}/finalize	Passage en facture définitive (FINAL)	200 OK / 409 Conflict
PATCH	/api/invoices/{id}/pay	Marquer la facture payée (PAID)	200 OK / 409 Conflict
PATCH	/api/invoices/{id}/cancel	Annulation avec motif	200 OK / 409 Conflict

↓ Consomme : MaintenanceCompleted

depuis Maintenance : réparation terminée
{maintenanceId, buildingId, estimatedLossM3Day, ...} → estimatedSavings mis à jour

Sécurité (API Gateway + JWT)

401 : jeton absent ou expiré · 403 : rôle insuffisant
customerId lu dans le jeton, aucun appel à User

Codes : 200 OK · 201 Created · 204 No Content · 400 données invalides · 404 introuvable · 409 action impossible selon le statut
503 Water Consumption ou Building indisponible (fallback Resilience4j) · format d'erreur commun : {timestamp, status, error, message, path}

GET : LISTER ET CONSULTER LES FUITES

POST : SIMULER UNE FUITE

PATCH : VÉRIFIER OU MARQUER COMME FAUSSE ALERTE

409 : ACTION IMPOSSIBLE SELON LE STATUT

ÉVÉNEMENTS : REÇOIT MEASUREMENTRECORDED ET MAINTENANCECOMPLETED, PUBLIE LEAKDETECTED

CONTRATS D'ÉVÉNEMENTS & GESTION DES ERREURS

CONSUME **MaintenanceCompleted**

`maintenanceId · buildingId · leakDetectionId`
`partReference · partQuantity · estimatedLossM3`

Menu de module de facturation

CONSUME **UserDeactivated**

`userId · role`

PUBLISH **InvoiceGenerated**

`invoiceId · buildingId · customerId · totalAmount · periodStart · periodEnd`

RÈGLES DU CONTRAT:

- · Événements versionnés, contenu minimal
- . Consommateur idempotent
- . Message rejoué toléré

Bâtiment introuvable

- La facture reste en statut DRAFT, l'association est retentée (REST GET vers Building COMMU).

Tarif introuvable

- Catégorie tarifaire par défaut appliquée. Log d'alerte pour l'administrateur.

Consommation introuvable

- Facture impossible. Retentatives avec exponential backoff. La période reste à facturer.

Transition interdite

Réponse 409 Conflict : le statut de la facture n'est pas modifié.

Message en double

Ignoré grâce à l'idempotence (une intervention = une seule estimation d'économie).

Échec de traitement d'un message

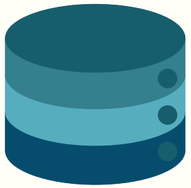
Nouvelles tentatives (Retry-Exchange), puis file d'erreurs (Dead-Letter Queue).

MICRO SERVICE
WATER CONSUMPTION SERVICE

WATER CONSUMPTION SERVICE : RÔLE MÉTIER & PÉRIMÈTRE

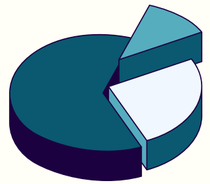
RÔLE UNIQUE & MÉTIER

Source de vérité unique des relevés de consommation d'eau. Enregistre les mesures, calcule les écarts d'index, conserve l'historique et gère la traçabilité des corrections.



POSTGRESQL (WATER_DB)

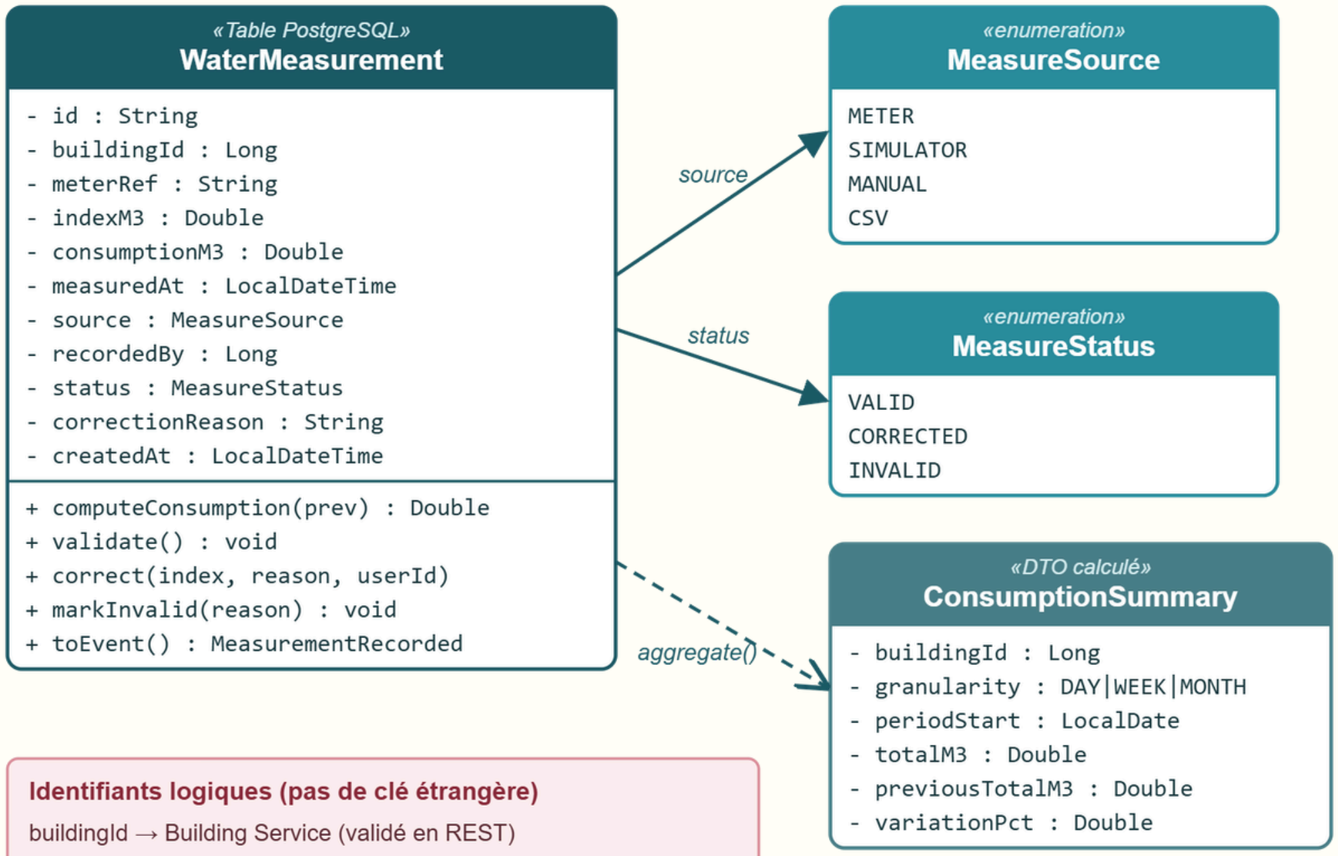
Choix de PostgreSQL (SQL) : schéma stable des relevés, contrainte UNIQUE (meterRef, measuredAt), agrégations SQL rapides pour Billing et transactions pour l'outbox.



CONTRIBUTION SERVICE COMMUN

Gestion des compteurs dans le Building Service (COMMUN) : enregistrement de meterRef, remplacement de compteur et traitement de l'événement BuildingUpdated.

CONCEPTION : DIAGRAMME DE CLASSES & RÈGLES MÉTIER

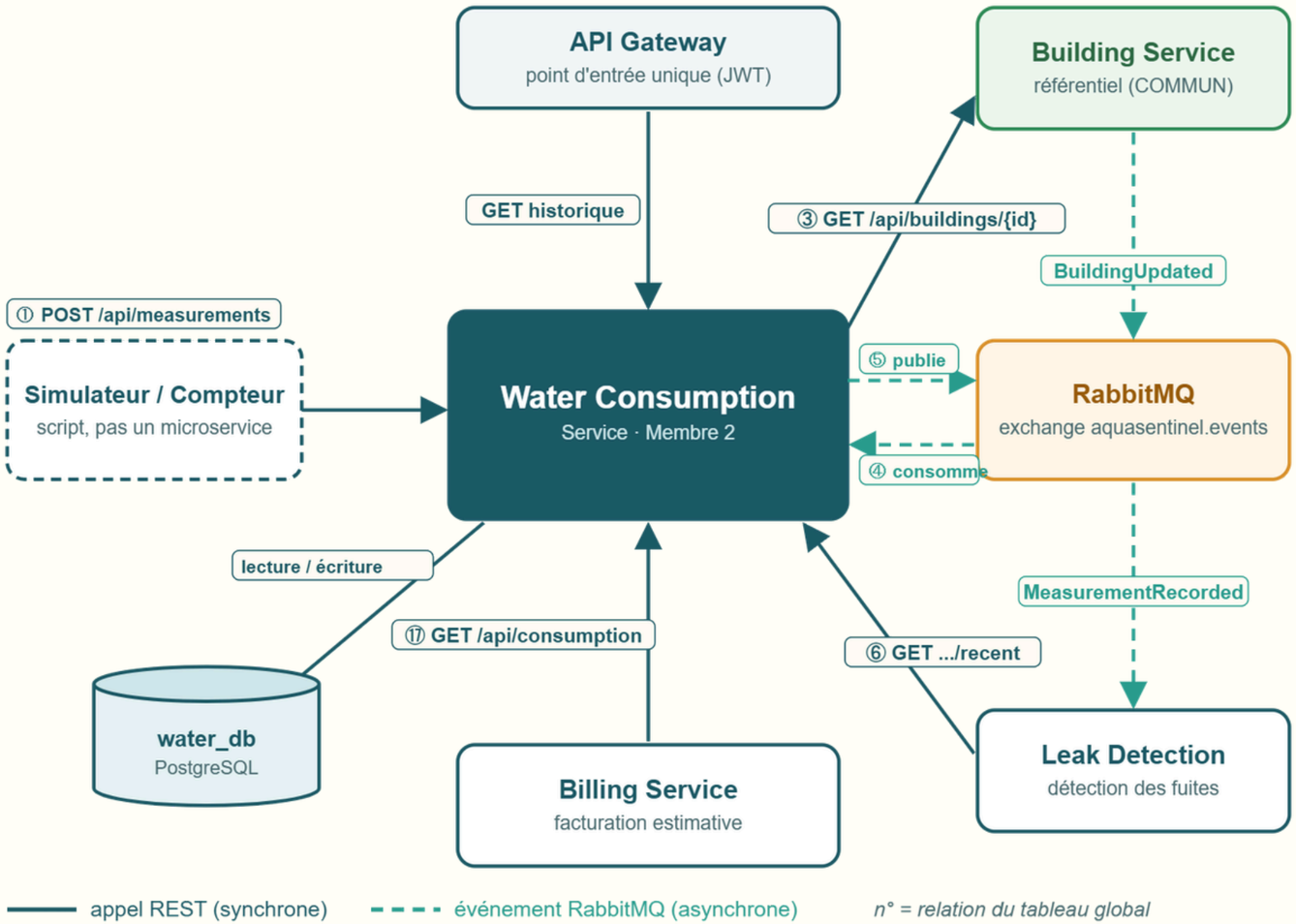


Identifiants logiques (pas de clé étrangère)

buildingId → Building Service (validé en REST)
meterRef → Building.meterRef (événement BuildingUpdated)
recordedBy → User.id (lu dans le jeton JWT)
Index unique : (meterRef, measuredAt) → idempotence

• Entité principale (WaterMeasurement) :
Un relevé horodaté d'un compteur : index cumulé (indexM3) et consommation calculée (consumptionM3 = index - index précédent).
• Origine (MeasureSource) :
METER, SIMULATOR, MANUAL ou CSV ; recordedBy garde la traçabilité de la saisie.
• Qualité (MeasureStatus) :
VALID par défaut, CORRECTED après correction avec motif obligatoire, INVALID si l'index diminue.
• Règles métier :
Bâtiment et compteur validés auprès de Building ; un seul relevé par (meterRef, measuredAt) ; seul un relevé VALID publie MeasurementRecorded.
• Agrégation (ConsumptionSummary) :
Calculée à la demande par jour, semaine ou mois, avec comparaison à la période précédente (variationPct).
• Méthodes clés :
computeConsumption(), validate(), correct(), markInvalid(), toEvent().

INTERACTIONS SYNCHRONES & ASYNCHRONES



• Communications synchrones (REST / OpenFeign) :

→ Building : GET /api/buildings/{id} pour valider le bâtiment et son compteur (meterRef) avant d'enregistrer un relevé.

→ Appelé par Leak Detection (historique récent du compteur) et par Billing (consommation d'une période).

• Communications asynchrones (RabbitMQ) :

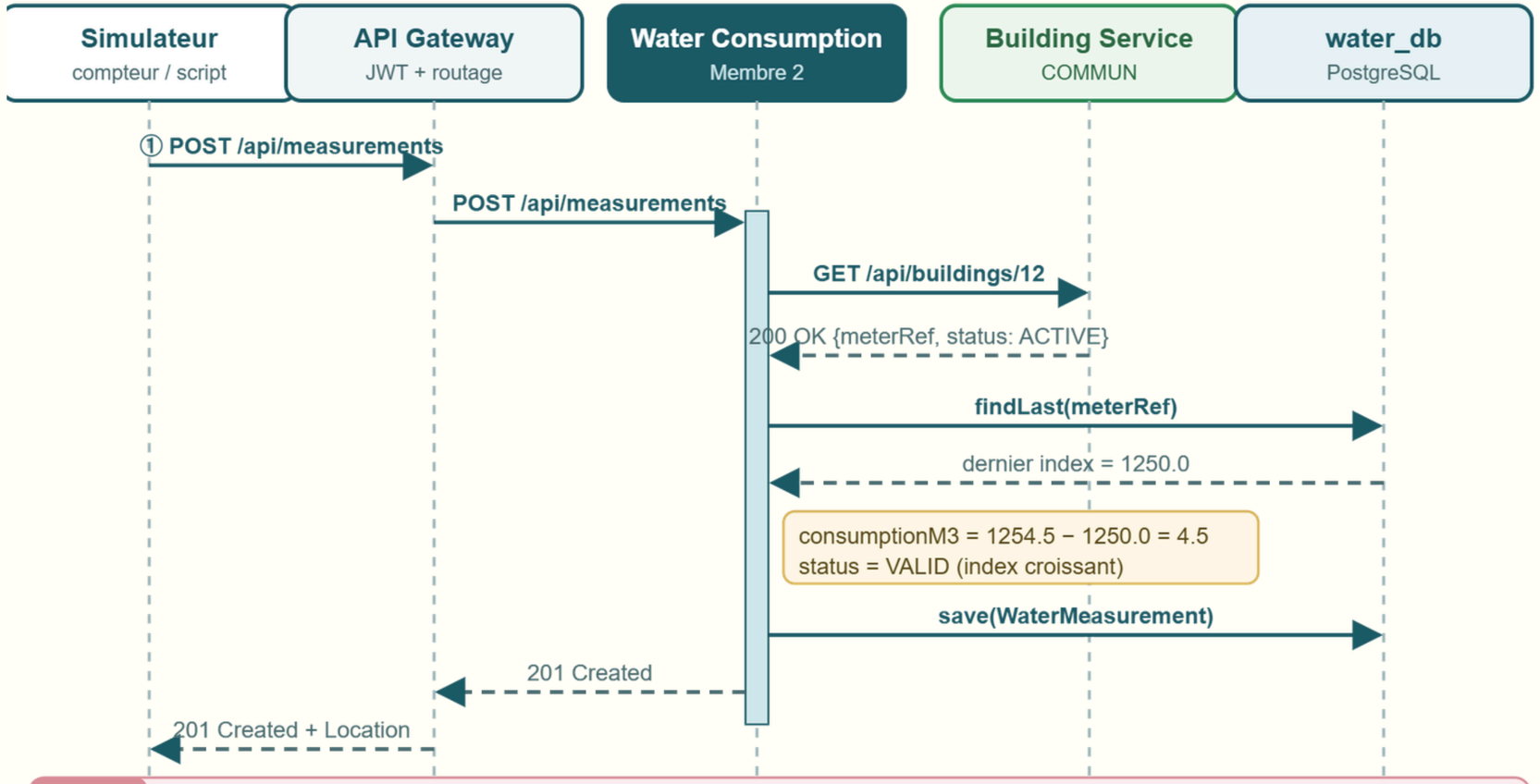
→ Publie MeasurementRecorded après chaque relevé VALID : Leak Detection analyse le nouveau relevé.

→ Consomme BuildingUpdated (Building) : prise en compte du nouveau compteur (meterRef) lors d'un remplacement.

• Tolérance aux pannes :

Building indisponible → 503 + retry avec timeout ; événement publié après la sauvegarde ; message rejoué traité de façon idempotente (measurementId).

EXEMPLE SYNCHRONE : ENREGISTRER UN RELEVÉ



[alt] **Cas d'erreur (aucun relevé enregistré, aucun événement publié)**

400 Bad Request : champ manquant / format invalide 404 Not Found : bâtiment ou compteur inconnu

409 Conflict : relevé déjà reçu (meterRef, measuredAt) 422 Unprocessable : index inférieur au précédent

503 Service Unavailable : Building ne répond pas (timeout 2 s, 3 tentatives, circuit breaker)

Requête — POST /api/measurements

```
{
  "buildingId": 12,
  "meterRef": "MTR-TUN-0012",
  "indexM3": 1254.5,
  "measuredAt": "2026-11-10T02:00:00Z",
  "source": "SIMULATOR"
}
```

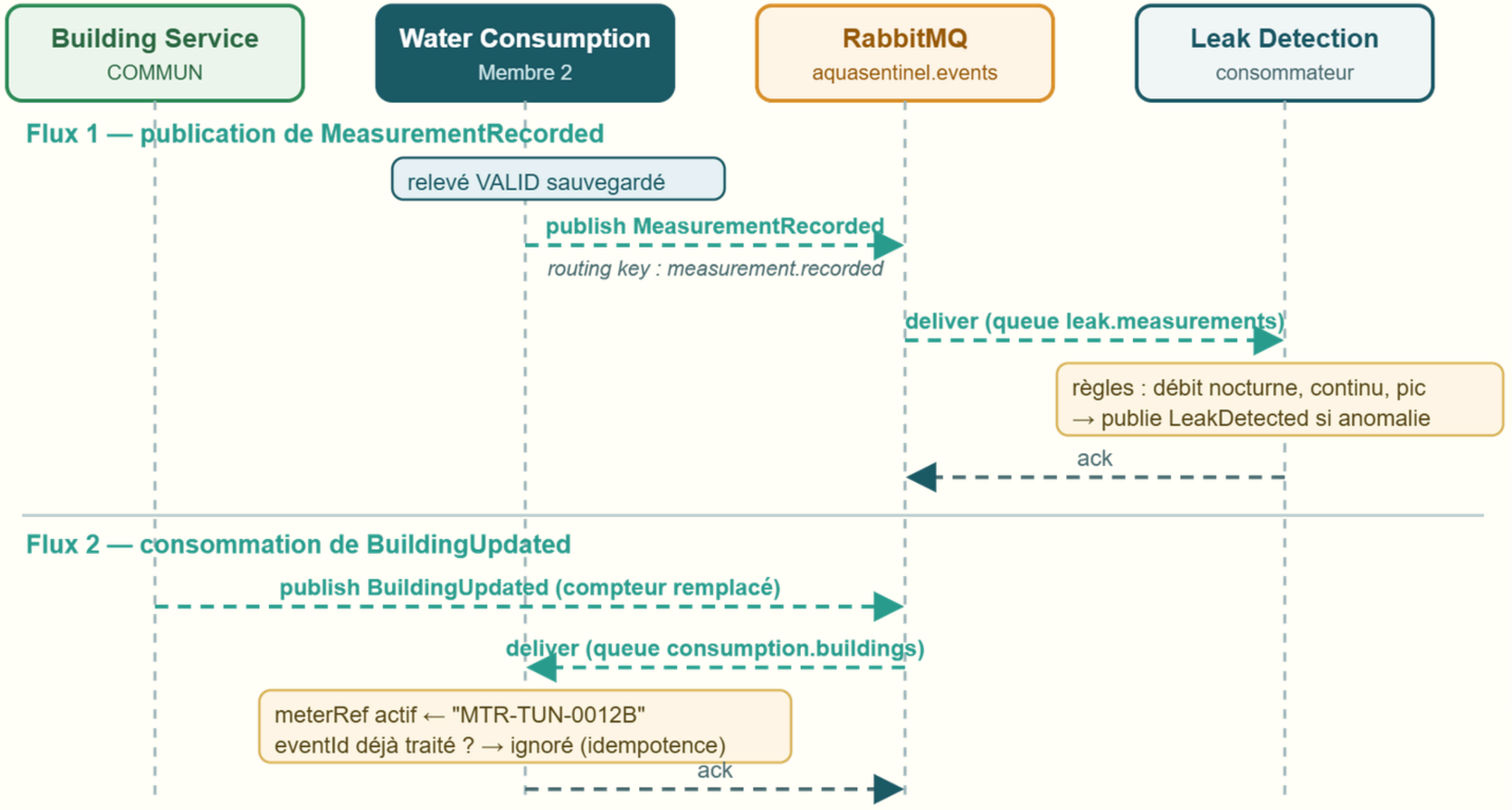
Réponse — 201 Created

```
{
  "id": "6731f0a2c9e1",
  "consumptionM3": 4.5,
  "status": "VALID"
}
```

2e exemple : Billing → Water Consumption

```
GET /api/consumption
    ?buildingId=12
    &from=2026-10-01&to=2026-10-31
// 200 OK
{ "totalM3": 38.7,
  "previousTotalM3": 41.2 }
```

EXEMPLE ASYNCHRONE : ÉVÉNEMENTS RABBITMQ



MeasurementRecorded v1 (publié)

```
{
  "eventId": "e-58c1d2",
  "type": "MeasurementRecorded",
  "measurementId": "6731f0a2c9e1",
  "buildingId": 12,
  "meterRef": "MTR-TUN-0012",
  "consumptionM3": 4.5,
  "measuredAt": "2026-11-10T02:00:00Z"
}
```

BuildingUpdated v1 (consommé)

```
{
  "eventId": "e-91ab07",
  "type": "BuildingUpdated",
  "buildingId": 12,
  "meterRef": "MTR-TUN-0012B",
  "status": "ACTIVE"
}
```

Consommateur de MeasurementRecorded

```
Leak Detection → analyse du relevé
(appel REST ensuite : historique récent)
```

Garanties de fiabilité

• Publication après la sauvegarde (pattern outbox) • livraison « au moins une fois » • consommateur idempotent (eventId)
• 3 tentatives puis Dead Letter Queue • événements versionnés (v1) avec le contenu minimal utile au consommateur

CONTRATS D'API ENDPOINTS & CODES DE RÉPONSE

Méthode	Endpoint	Description & rôle métier	Codes de réponse
POST	`/api/measurements`	Enregistrer un relevé (calcul de la consommation)	201 400 404 409 422
POST	`/api/measurements/import`	Import CSV de relevés (traitement en lot)	202 400
GET	`/api/measurements?buildingId&from&to&page`	Historique paginé (bâtiment, compteur, période)	200 400
GET	`/api/measurements/{id}`	Consulter un relevé	200 404
PUT	`/api/measurements/{id}/correction`	Corriger un relevé avec motif (→ CORRECTED)	200 400 404 422
DELETE	`/api/measurements/{id}`	Supprimer un relevé de test (ADMIN)	204 403 404
GET	`/api/measurements/meter/{meterRef}/recent`	Historique récent du compteur (Leak Detection)	200 404
GET	`/api/consumption?buildingId&from&to`	Consommation d'une période (Billing)	200 400 404
GET	`/api/consumption/aggregate?granularity=MONTH`	Agrégation jour/semaine/mois + période précédente	200 400

↑ Publie : MeasurementRecorded
vers Leak Detection, après chaque relevé VALID
{measurementId, buildingId, meterRef, consumptionM3, …}

↓ Consomme : BuildingUpdated
depuis Building : changement de compteur
{buildingId, meterRef, status} → meterRef actif mis à jour

Sécurité (API Gateway + JWT)
401 : jeton absent ou expiré · 403 : rôle insuffisant
écriture : ADMIN / MANAGER · lecture : tous les rôles

Codes : 200 OK · 201 Created · 202 Accepted · 204 No Content · 400 données invalides · 404 introuvable · 409 doublon (meterRef, measuredAt)
422 règle métier violée (index qui diminue, motif absent) · 503 Building indisponible · format d'erreur commun : {timestamp, status, error, message, path}

CONTRATS D'ÉVÉNEMENTS & GESTION DES ERREURS

↑ MeasurementRecorded v1 — publié

exchange aquasentinel.events · routing key measurement.recorded · → Leak Detection

Champ	Type	Description
`eventId`	UUID	identifiant unique (idempotence)
`measurementId`	String	relevé source (WaterMeasurement.id)
`buildingId`	Long	référence logique vers Building
`meterRef`	String	compteur qui a produit le relevé
`consumptionM3`	Double	écart avec le relevé précédent
`measuredAt`	DateTime	date et heure de la mesure

↓ BuildingUpdated v1 — consommé

producteur : Building Service · queue consumption.buildings

Champ	Type	Description
`eventId`	UUID	identifiant unique (idempotence)
`buildingId`	Long	bâtiment concerné
`meterRef`	String	nouveau compteur actif
`status`	Enum	ACTIVE / INACTIVE

Effet : compteur actif mis à jour ; ancien meterRef refusé (404).

Format d'erreur commun (exemple : 422)

```
{
  "timestamp": "2026-11-10T02:00:03Z",
  "status": 422,
  "error": "Unprocessable Entity",
  "message": "indexM3 1249.0 < index précédent 1250.0",
  "path": "/api/measurements"
}
```

Résilience

- Appel Building : timeout 2 s + retry, puis 503
- Pattern outbox : aucun événement perdu
- eventId déjà traité → message ignoré
- Échecs répétés → Dead Letter Queue

MERCI